CallVerse

Multi-agent digital twin of an e-commerce customer-service centre

Technical guide — every concept, every component and every page of the front office and the back office, explained with real screenshots.

Project	Integrated project (Advanced Data Science) — ESPRIT
Author	Hamza Farhani
Version	1.1 — Hugging Face LLMs, front office + back office
Date	October 2026
Source	branch `hamza` of the CallVerse repository

Contents

1. **The project in brief** — problem, objectives, architecture, results
2. **How everything works** — data, forecast, queueing theory, the digital twin, the Workforce agent (CP-SAT + PPO), the conversational agents, the Hugging Face LLMs, LangGraph and live sessions, evaluation, API, frontend, MLOps
3. **Front office, page by page** — home and semantic search, help centre, demo customer, live chat with the AI advisor, advisor desk
4. **Back office, page by page** — overview, data, forecast, digital twin (live floor + scenario lab), agent theater, workforce, quality, scorecard, models & MLOps
5. **End-to-end walkthroughs** — one customer message from keyboard to quality review; one what-if from click to confidence interval
6. **Appendix** — glossary, commands, folder map, honest limits

> **How to read this guide**
>
> Part 2 explains each concept once, with the formulas and the design decisions. Parts 3 and 4 then walk through every screen: each screenshot is followed by a numbered list that names every element (cards, charts, badges) and says what it shows, where the number comes from and how to read it. All screenshots are real: they were captured from the running apps (API with the local models on the GTX 1650 GPU), during live conversations and simulations.

> **About the numbers**
>
> Forecast, twin, scheduling, routing and BO4 numbers come from `callverse evaluate` (10 seeds, 95% CIs). The DSO2 conversation table (verified actions, CSAT, judge kappa) was measured with the first LLM stack (Ollama qwen2.5/llama3.2); the re-run with the Hugging Face models (`callverse agents`, `callverse judge-calibrate --judge llm`) is the next planned step. Retrieval numbers do not depend on the LLM.

PART 1

The project in brief

What problem CallVerse solves, how the business objectives become data-science objectives, and how the five layers fit together.

1.1 The problem

An e-commerce contact centre fails in two ways: **too few advisors** during promotions or delivery incidents (customers wait, hang up, complain) and **too many** when it is quiet (money wasted). Decisions — how many people to schedule, who takes which customer, what an advisor may promise — are usually made from intuition and tested directly on real customers.

CallVerse is a digital twin: a virtual copy of the contact centre where every decision is tested first. It forecasts how many customers will contact the centre, plans the shifts, simulates every customer and every advisor minute by minute, lets AI agents answer and grade conversations, and compares every idea with a baseline using confidence intervals.

1.2 From business objectives to data-science objectives

The framework follows a strict chain: **BO** (business objective) → **KPI** → **DSO** (data-science objective) → metric. Each DSO has a *main idea*, a *baseline* it must beat, a *backup*, and a *switch rule* that decides automatically which one is kept.

Objective	Main idea	Baseline	Backup	Switch rule
BO1 / DSO1 shorter waits	Reinforcement-learning routing (MaskablePPO)	FIFO with skill matching	Tuned skill-score router	≥ 10% less waiting than FIFO in the twin's peak weeks
BO2 / DSO2 correct answers, satisfaction	Agentic RAG advisor + LLM judge	Keyword FAQ search / templates	Intent classifier + approved templates; checklist reviewer	≤ 10% wrong answers; judge agreement kappa ≥ 0.4
BO3 / DSO3 lower cost, same service	Forecast + OR-Tools CP-SAT shift planning	Fixed, peak-sized schedule	Erlang C + shift templates + overtime rule	Cost −5% or better while the 80/20 service target is met; labour rules respected
BO4 / DSO4 prepare for peaks	Intent-level LightGBM forecast + SimPy digital twin	"Same day last week"; Erlang C queue model	Prophet/SARIMA + Erlang C	Beat the naive forecast; twin passes the fidelity checks

1.3 Results at a glance

−19.9%
forecast error (wMAPE 25.3% vs 31.6%)

−40.1%
labour cost, service level 92.4%

−60 to −84%
understaffed hours when the plan is tested first

96.9%
retrieval recall@4 (vs 80.4% keyword)

Objective	Verdict	Why
BO4 / DSO4	**main idea kept**	LightGBM beats every baseline on the total; the twin passes every fidelity check (KS p = 0.998, Erlang C error < 4%); preparing for a peak saves €730 net over three peak weeks.
BO3 / DSO3	**main idea kept**	CP-SAT plan: −40.1% labour cost vs the fixed schedule, service level 92.4% (target 80%), all labour rules met.

BO1 / DSO1	**backup kept**	PPO cuts the average speed of answer by 13.9% on unseen stressed days, but not the mean wait in full peak weeks (+2.2%) → the tuned skill-score router is the production candidate.
BO2 / DSO2	**hybrid kept**	Retrieval clearly beats keyword search, but a small LLM alone invents order ids and promises → "rules engine decides, LLM writes"; the LLM judge is not yet trusted (kappa < 0.4).

1.4 Architecture: five layers



Each layer is a Python package in `src/callverse/`. A lower layer never imports a higher one: this is enforced by five `import-linter` contracts checked in CI (for example, *data* may not import *agents*, *evaluation* may not import *api*). Every assumption (contact rates, opening hours, costs, model names) lives in a YAML file in `configs/`.

1.5 Two applications on one API

App	Users	What it is for
FRONT OFFICE localhost:5173	Customers and advisors	The "shop side": a help centre with semantic search, a demo customer account with the order journey, a **live chat with the AI advisor** (tokens stream, every agent step is visible), and an **advisor desk** where a human answers simulated customers with an AI copilot.
BACK OFFICE localhost:5174	Managers, data scientists, the jury	The control room of the twin: overview and scorecard, data, forecast, an **animated live floor** replaying a simulated day, the scenario lab, an **agent theater** where the four agents run alone, workforce planning and routing, judge calibration, models and MLOps.
API localhost:8000	Both apps	FastAPI: REST endpoints + two WebSockets; Swagger documentation at `/docs`.

PART 2

How everything works

Every concept used in CallVerse, in the order data flows through the system: from raw orders to a forecast, from a forecast to a staffing plan, from a plan to a simulated day, and from a customer message to a graded AI answer.

2.1 Data layer

Why synthetic contacts

No public dataset contains real contact-centre logs linked to orders. CallVerse therefore starts from two real public datasets and generates the contacts with **explicit, documented assumptions** (all marked `is_synthetic = True`):

- **Olist** — 99,441 real orders of a Brazilian marketplace (2016–2018): purchase, approval, carrier and delivery timestamps, estimated delivery date, payments, items, reviews, customer city.
- **Bitext** — real customer-service utterances labelled by intent. 17 order-related intents are kept and grouped into four **problem families**: *wismo* (where is my order), *cancel_refund*, *post_delivery* (damaged, wrong, missing items, returns) and *order_admin* (invoices, address changes, payment).

How a contact is generated

1. Each order is inspected for **situations** that make customers write: delivery late, seller shipped late, order canceled, payment anomaly, low review score, long transit…
2. Each situation has a **contact probability** and a timing (days after an anchor event, e.g. 2 days after the shipping limit), set in `configs/data.yaml`. A random draw (fixed seed) decides whether the customer contacts the centre.
3. The contact receives an **intent** consistent with the situation and the **real wording** of a Bitext message of that intent.
4. Promotion windows (Black Friday, Christmas) and public holidays shift the volumes.

Result: **9,179 contacts** (Jan 2017 → Aug 2018), each linked to a real order. From them, **9,179 tickets** and **60,837 chat messages** are generated: a rules engine decides the policy-correct *outcome* (tracking shared, refund approved, carrier investigation, hand-over to logistics…), approved answer templates write the advisor turns, and a 30% CSAT survey is simulated.

SQLite store and knowledge base

- **SQLite** (`data/callverse.db`) holds orders, tickets and chats with keys; the advisor's tools query it "as of" the moment the customer writes, so an order that was "shipped" on that day is never shown as "delivered".
- **Knowledge base**: 10 hand-written policy documents (refunds, returns, delays, invoices, payments, cancellations, tracking, FAQ, damaged items, advisor playbook). Each "##" section becomes a chunk (58 chunks), embedded with **bge-m3** (multilingual, 1024-dimension vectors) and stored in a **FAISS** exact inner-product index. Vectors are L2-normalised, so the inner product equals the **cosine similarity**.
- **Pandera contracts** check every processed table (types, ranges, uniqueness, allowed values) after each pipeline run, before the forecast and the twin read the data.
- The data notebooks (03–08) stay the single source of truth: `callverse data` re-executes them headless (nbclient).

2.2 Demand forecast (DSO4, part 1)

Target: **daily number of contacts per problem family**, forecast 1 to 7 days ahead (the planning horizon of a weekly roster).

Features — leakage-safe

Every lag is **≥ 7 days**, so a forecast made on Sunday for the next Saturday only uses information available on Sunday: contacts 7, 14, 21, 28 days before; orders, deliveries, late deliveries and overdue backlog with the same lags; day of week, promotion calendar, public holidays. A unit test checks that `lag_7` equals the value 7 days earlier.

Model and baselines

- **Main**: one **global LightGBM** model for the four groups (the group is a feature), with a **Poisson** objective because the target is a count.
- **Baselines**: *seasonal naive* ("same day last week", the DSO4 baseline), 4-week weekday mean, **SARIMA**, **Prophet**.

Rolling-origin backtest

Every Sunday from October 2017 to August 2018 (46 origins), each model is trained only on the past and forecasts the next 7 days. Errors are aggregated over all origins:

$$\text{wMAPE} = \Sigma\,|\text{actual} - \text{forecast}|\,/\,\Sigma\,\text{actual} \quad \text{MAE} = \text{mean}\,|\text{error}| \quad \text{RMSE} = \sqrt{\text{mean}(\text{error}^2)} \quad \text{bias} = \Sigma(\text{forecast} - \text{actual})\,/\,\Sigma\,\text{actual}$$

wMAPE is the decision metric: unlike MAPE it does not explode on days with very few contacts. Result on the total: **LightGBM 25.3%** vs 31.6% (naive), 27.7% (SARIMA), 31.9% (4-week mean), 35.9% (Prophet). Small groups (a few contacts a day) are close to the *Poisson noise floor*, so the total is the decision level. Every run is logged to **MLflow**.

2.3 Queueing theory: Erlang C

A contact centre is a queue with *c* servers (advisors). With arrival rate λ (contacts per hour) and average handle time AHT, the offered load is **A = λ · AHT** (in Erlangs: the number of advisors that would be busy all the time). The Erlang C formula gives the probability that a customer has to wait:

$$P(\text{wait}) = [\ A^c/c! \cdot c/(c-A)\ ] \ /\ [\ \Sigma_{k=0..c-1}\ A^k/k! + A^c/c! \cdot c/(c-A)\ ]$$

$$\text{Service level} = 1 - P(\text{wait}) \cdot e^{-(c-A)\cdot T/AHT} \qquad ASA = P(\text{wait}) \cdot AHT\ /\ (c - A)$$

CallVerse uses it three ways: (1) the **required number of agents** per hour is the smallest *c* reaching the **80/20 target** (80% of chats answered within 20 s) with occupancy ≤ 90%; (2) the Workforce agent quotes the **expected wait** of a conversation; (3) it **validates the simulator**: on a stationary hour the SimPy engine must reproduce Erlang C (error < 4%).

Little's law (L = λ · W: average number waiting = arrival rate × average wait) explains why the routing reward "minus customer-seconds of waiting" is the same as minimising the average wait.

2.4 The digital twin (DSO4, part 2)

The twin is a **discrete-event simulation** written with **SimPy**: time jumps from event to event (an arrival, an advisor becoming free, a customer hanging up, a shift starting) instead of ticking every second.

Ingredients

Element	How it is modelled
Arrivals	Daily volumes per group from the history or the scenario, spread over the day with the hourly profile, as a Poisson process (random but realistic bursts).
Live vs asynchronous	Chats arriving during opening hours (8:00–22:00) are **live**; messages arriving at night form an **overnight backlog** worked first in the morning.
Customers	Personas: **impatient 25%** (patience ×0.5), **normal 55%**, **patient 20%** (×2). A customer whose patience runs out **abandons** (hangs up). Repeat contacts are more often impatient.
Handle time	Lognormal per problem family (long right tail).
Advisors	Each has a **main skill**: ×0.75 handle time on their specialty, ×1.3 on weak groups. A **duty matrix** (agent × hour) says who works when.
Overtime	Planned overtime hours, plus a real-time rule: if the queue exceeds 8 customers, 2 on-call agents join.
Routing	A **router** decides which waiting customer a free advisor takes: FIFO, skill score or PPO (section 2.6).
Costs	€12/h regular, overtime ×1.5, €8 per lost (abandoned) customer.

KPIs

Service level (answered within 20 s), ASA (average speed of answer), mean wait of all live customers, p90 wait, abandon rate, occupancy, understaffed hours (scheduled < Erlang C requirement), overtime hours, labour cost, cost per contact, total cost.

Scenarios and "prepared vs unprepared"

Scenarios (`configs/scenarios.yaml`) are real weeks with multipliers: *promo spike*, *delivery delays*, *staff shortage* (absences), *Black Friday 2017*. A plan is **unprepared** when it is built from the normal forecast and **prepared** when it is built after testing the scenario in the twin. Comparing both on the same customers measures the value of testing first (BO4).

Statistics: common random numbers and paired confidence intervals

Each option is simulated over **10 seeds**. For a given seed, every option sees **exactly the same customers** (same arrival times, same patience, same handle times): this is the *common random numbers* technique. The difference between two options is computed seed by seed, and a **paired t-interval** gives its 95% confidence interval. A difference is "significant" when the interval does not contain 0. Pairing removes the noise of the customers themselves, so small effects become measurable.

Fidelity: is the twin realistic?

- **Kolmogorov–Smirnov test** on daily volumes, simulated vs historical: p = 0.998 (no detectable difference).
- **Hourly profile**: total-variation distance 0.015 between simulated and historical shares per hour.
- **Erlang C check**: on stationary hours, simulated P(wait) and ASA within 4% of the formula.

2.5 Workforce agent — shift planning (DSO3)

Given the hourly requirement (Erlang C on the forecast), the planner chooses **who works when**. It is an integer optimisation solved with Google **OR-Tools CP-SAT** in two stages:

1. **Coverage**: choose how many people start each of 27 **shift templates** (4, 6 or 8 hours) each day, plus overtime hours (≤ 4 per hour, ≤ 60 per week), minimising *regular cost + 1.5 × overtime cost + 200 × understaffed agent-hours*.
2. **Roster**: assign the chosen shifts to named agents under hard **labour rules**: at most 5 days and 40 hours a week, one shift per day, at least 11 hours of rest between shifts.

The solver proves optimality in under a second. It runs with `num_workers = 1`: with 8 parallel workers it returned different (equally optimal) rosters from run to run, which moved the simulation KPIs — determinism matters more than speed here.

Baselines and backups: *fixed schedule* sized for the peak (same staff every hour), *fixed tuned* (sized at the 75% quantile), *Erlang C + rules* (morning/afternoon templates + overtime rule), *Erlang C hourly* (theoretical reference).

2.6 Workforce agent — real-time routing (DSO1)

When an advisor becomes free, the router chooses which waiting customer they take.

- **FIFO with skill matching** (baseline): oldest customer first.
- **Skill score** (backup): a weighted score of waiting time, the advisor's speed on that problem, persona and live/overnight; the weights are tuned by random search in the twin.
- **MaskablePPO** (main idea): a reinforcement-learning policy trained only inside the simulator.

The RL problem (MDP)

Part	Definition
State	The **10 oldest waiting customers**, 12 features each (wait so far, problem family, persona, live or overnight, the free advisor's speed on that family…) + 7 global features (queue length, staff on duty, time of day…).
Action	Which of the 10 slots the free advisor takes. Empty slots are **masked** (MaskablePPO never picks an invalid action).
Reward	−(Δ customer-seconds of waiting + 600 × Δ abandons) / 600: every waiting second costs, a hang-up costs 10 minutes.
Training	300,000 steps on single busy days (load ×1.0–1.8, group surges, 5–30% absences), 2017-02 → 2018-03; test on unseen days 2018-04 → 2018-08. Network 2 × 128, γ = 0.995, learning rate 3e-4, entropy 0.01.

The SimPy engine has a **steppable mode** (`advance()` / `resolve()`) wrapped as a **Gymnasium** environment, so the same simulator serves training, evaluation and the live floor.

Result: on unseen stressed days PPO reduces ASA by 13.9% versus FIFO; in full peak weeks (multi-day backlog, planned overtime) it does not reduce the mean wait (+2.2%) although it cuts long waits (p90 −18 to −25%). Following the switch rule, the tuned skill-score router is the production candidate; PPO stays the research track (next step: multi-day training episodes).

2.7 The conversational agents

Client agent (plays the customer)

It knows a **profile**: the real opening message, the order, the hidden situation, a persona and a tone. It answers the advisor with an LLM in JSON (`{"message", "end", "csat"}`) or with rules offline. Guardrails: verification questions ("your order number?") are always answered deterministically, and if the LLM starts writing like an advisor ("how can I help you?") the turn falls back to the rule-based customer. At the end it rates the chat 1–5 (CSAT).

Advisor agent: Ask → Retrieve → Decide → Reply

1. **Understand**: an intent classifier (TF-IDF + logistic regression, 99.0% accuracy on held-out Bitext) predicts the intent.
2. **Retrieve**: bge-m3 + FAISS returns the top-3 policy sections. Below a cosine of 0.57 the advisor says the knowledge base has no answer.
3. **Verify**: no order data is shared before the customer gives an order number (playbook rule 1).
4. **Tools** on the SQLite store, as of the contact time: `lookup_order`, `get_ticket_history`, `check_refund_eligibility`, `open_ticket`, `search_knowledge_base` and `recommend_resolution` (the company rules engine).
5. **Reply** in plain language.

Variant	How it works
template (backup)	Classifier + approved answers filled with the order facts. Deterministic, instant, used offline.
rag (pure idea)	The LLM decides which tools to call and what to answer.
rag_policy (hybrid, production)	As soon as the order number is known, the pipeline itself calls `lookup_order` and the rules engine; the LLM receives the decision ("action: carrier investigation, hand over to: logistics, approved answer: …") and only **writes** it. *The rules engine decides, the LLM writes.*

Guardrails added after the first LLM runs: **tool gating** (order tools exist only after an order number appears — small models called them with invented ids), and a **verification guardrail** (a reply given before verification always ends by asking for the order number).

Quality agent (the reviewer)

It fills a **scorecard** of five criteria (1–5): *compliance* (policy followed, no invented promise), *relevance*, *satisfaction*, *sentiment*, *resolution*, plus "correct" and "resolved" verdicts and a one-sentence rationale. It receives the **policy-correct reference** (what the rules engine says for the true intent). The LLM judge comes from a **different model family** than the advisor (IBM Granite vs Hugging Face SmolLM3) so it does not share its blind spots. Offline, a rule-based checklist fills the same scorecard.

Can the judge be trusted? Calibration

The judge's grades are compared with labels using **Cohen's quadratic kappa** (chance-corrected agreement on an ordinal scale, 1 = perfect, 0 = chance; disagreements of 2 points cost 4× more than 1 point) and **Spearman's ρ**. Rule: trusted at kappa ≥ 0.4. Without human labels, proxy labels come from the generator's answer key; once 20+ chats are graded by humans on the labeling page, the calibration switches to them automatically.

Workforce agent inside a conversation

For each chat it predicts the problem family, picks the best-skilled desk and quotes the Erlang C expected wait for that hour.

2.8 The local LLMs (Hugging Face)

Role	Model	Why
Advisor + simulated customer	**SmolLM3-3B** (Hugging Face), 3.1 B parameters, Apache-2.0	Fully open (weights, data, recipe), tool calling, strong for its size; one copy serves both roles.
Quality judge	**Granite-4.0-1B** (IBM), 1.6 B parameters, Apache-2.0	Different family and organisation than the advisor (no self-preference bias); small enough to stay loaded.
Retrieval	**bge-m3** (BAAI), on CPU	Multilingual embeddings; keeps the GPU for the LLMs.

Fitting two LLMs in a 4 GB laptop GPU

- **4-bit NF4 quantization** (bitsandbytes): each weight is stored in 4 bits using "NormalFloat" levels that match the bell-shaped distribution of neural-network weights; *double quantization* also compresses the scaling constants. A 3B model goes from ~6 GB (16-bit) to ~2 GB. Both models together use **~3.2 GB** of the GTX 1650's 4 GB.
- **bfloat16 compute**: the matrix products run in bf16. Float16 overflows on these models (activations above 65,504) and the output becomes "!!!!"; bf16 has the range of float32, and PyTorch emulates it on this Turing GPU at the same speed.
- Speed: ~8–13 tokens/s; an advisor reply takes ~7–10 s, a whole four-agent conversation ~45 s.

From messages to tokens and back

- **Chat templates**: each model has its own prompt format; `apply_chat_template` renders it. Tool schemas are passed the model's way (SmolLM3 uses `xml_tools`); system messages are merged into one; "thinking" mode is disabled for faster answers.
- **Tool-call parsing**: the model writes `<tool_call>{"name": …, "arguments": …}</tool_call>`; the parser also accepts Granite's and bare-JSON formats and ignores unknown tools.
- **Streaming**: a background thread generates while a `TextIteratorStreamer` yields text pieces; a **stream gate** forwards them to the UI but hides tool-call markup (it holds back the last 12 characters until it is sure they do not start a tool call).
- **JSON outputs** (client turns, judge scorecards): generation stops as soon as the JSON object is closed; a repair step closes an object that a small model left unfinished.
- **Cache**: every call is keyed by the SHA-256 of (model, commit, messages, tools, temperature, seed); an identical call is answered from SQLite instantly, which makes runs reproducible.
- **Tracing**: every call is appended to `llm_calls.jsonl` (agent, model, tokens, latency, tokens/s, tools, cache hit).
- **Pinning**: `callverse llm-pull` downloads the models and writes the exact Hub commit of each repository to `llm_models.lock.json`; afterwards models load from the local cache only.
- **Graceful degradation**: with no CUDA GPU or no downloaded weights, every agent switches to its offline backup and the UI says so.

2.9 Orchestration: LangGraph and live sessions

The automatic conversation graph



LangGraph runs this as an explicit state machine; the conditional edge after *client_reply* loops back to the advisor until the customer leaves (or 4 customer turns). The final **KPI row** holds: first-contact resolution, CSAT, compliant (compliance ≥ 4), policy-correct action (advisor's outcome = reference outcome), judge verdict, tool success rate, intent predicted, turns, expected wait. Batches of conversations (`callverse agents`) produce the DSO2 tables.

Live sessions: a human on either side

`orchestration/live.py` drives the **same agents** one step at a time, so a human can play the customer (front-office chat) or the advisor (advisor desk, with the AI as copilot or autopilot). The browser talks to it through the WebSocket `/ws/session` :

Event (server → UI)	Meaning
`ready`	Session created: profile, whether the LLM is used, which model, which retriever.
`stage`	An agent started a step: routing, retrieval (with hits and scores), tool (name, ok, latency), generation (model, round), typing, reviewing.
`token`	A piece of the advisor's text as it is generated (for a reply or a copilot draft).
`message`	A finished message (customer or advisor) with its metadata (mode, tools, sources).
`routing` , `advisor_state`	Workforce decision; policy state (intent, order verified, action, hand-over).
`suggestion`	The copilot's draft for the human advisor.
`review` , `done`	Quality scorecard, policy reference and KPI row; end of session.

The UI sends `start` , `customer_message` , `advisor_message` , `suggest` , `autopilot` and `end` (with the CSAT). The server runs each blocking agent step in a worker thread and pushes events through an asyncio queue; a global lock ensures one generation at a time on the GPU. Live transcripts are saved apart from the evaluation batches.

2.10 Evaluation framework

- **KPI engine** (`evaluation/business_kpis.py`): means, 95% CIs and paired differences of every KPI.
- **Experiments**: DSO3 planners vs fixed; DSO1 routers vs FIFO (stressed days and peak weeks); BO4 prepared vs unprepared; DSO2 retrieval (255 Bitext questions: recall@1, recall@4, MRR), conversations per variant, judge calibration, faithfulness and similarity to the policy answer.
- **Component metrics**: is each part good on its own (forecast wMAPE, retrieval recall, judge kappa, twin KS and Erlang C error)?
- **Scorecard**: applies each DSO's switch rule and writes the verdicts (`scorecard.json`), shown in both apps.
- **BO4 money**: saving = (unprepared − prepared) labour cost + €8 × lost customers avoided; the **break-even** value of a lost customer is the value above which preparing pays off (€6–12 depending on the scenario).

$$\text{MRR} = \text{mean}(1 \,/\, \text{rank of the first correct section}) \qquad \text{recall@k} = \text{share of questions whose correct section is in the top k}$$

2.11 API (FastAPI)

Endpoint	Used by — purpose
`/api/portal/customers`, `/api/portal/customers/{id}`	Front office — demo customers (one real contact per main intent), order journey "as of" the contact date, previous contacts.
`/api/kb/articles`, `/api/kb/articles/{id}`, `/api/kb/search?q=`	Front office — help centre (the advisor's knowledge base) and semantic search.
`/api/desk/queue`	Front office — contacts waiting for a human advisor.
`WS /ws/session`	Front office + agent theater — live sessions (section 2.9).
`/api/overview`, `/api/data/summary`, `/api/forecast`	Back office — headline numbers, data charts, backtest and next-week forecast.
`/api/twin/config`, `POST /api/twin/run`, `POST /api/twin/compare`, `/api/twin/floor`	Back office — scenarios, one configuration over seeds, several options with common random numbers, one day contact by contact.
`/api/workforce/schedule`, `/api/workforce/routing`	Back office — CP-SAT plan and roster; router comparison and PPO curve.
`/api/quality/*`, `/api/evaluation`, `/api/mlops`, `/api/system`	Back office — labeling and calibration, scorecard tables, MLflow runs and traces, models and GPU (+ `POST /api/system/warmup`).

2.12 Frontend architecture

- **npm workspaces**: `frontend/shared` (design system), `front-office`, `back-office`. Both apps are React 18 + TypeScript + Vite; the shared package is consumed as source, so a component change shows in both apps at once.

- **Design tokens**: colours are CSS variables with a light and a dark set; Tailwind classes point to them, so themes switch without rebuilding. Chart colours use a validated colour-blind-safe palette with a fixed colour per entity (e.g. "Where is my order" is always blue).
- **Motion** (Framer Motion): page transitions, staggered entrances, count-up numbers, animated rings and bars, spring chat bubbles, an animated agent pipeline. The OS "reduce motion" setting is respected; `?snapshot` renders every animation in its final state.
- **Live floor**: drawn on an HTML `<canvas>` at 60 fps (hundreds of moving customers would be too slow as DOM nodes).
- **Live session client**: a React hook (`useLiveSession`) turns WebSocket events into state (messages, streaming text, active agent, stages, routing, suggestion, review) through one reducer.
- **Quality of life**: command palette (Ctrl+K), toasts, skeleton loaders, error boundaries (one broken view never blanks the app), self-hosted fonts (works offline on demo day).

2.13 MLOps and software quality

- **MLflow** (SQLite backend locally, Postgres in Docker): forecast backtests, PPO training, evaluation suites — parameters, metrics, artifacts.
- **Reproducibility**: seeds in configs, pinned model commits, LLM cache, CP-SAT single worker, `callverse all` rebuilds everything.
- **Docker Compose**: api, web (one nginx serving the front office on 8080 and the back office on 8081, proxying `/api` and `/ws`), mlflow + postgres; `docker-compose.gpu.yml` adds CUDA PyTorch and the NVIDIA GPU.
- **CI (GitHub Actions)**: ruff, import-linter (5 layer contracts), pytest (95+ tests: unit, smoke, integration including the live WebSocket session), type-check and build of both apps.

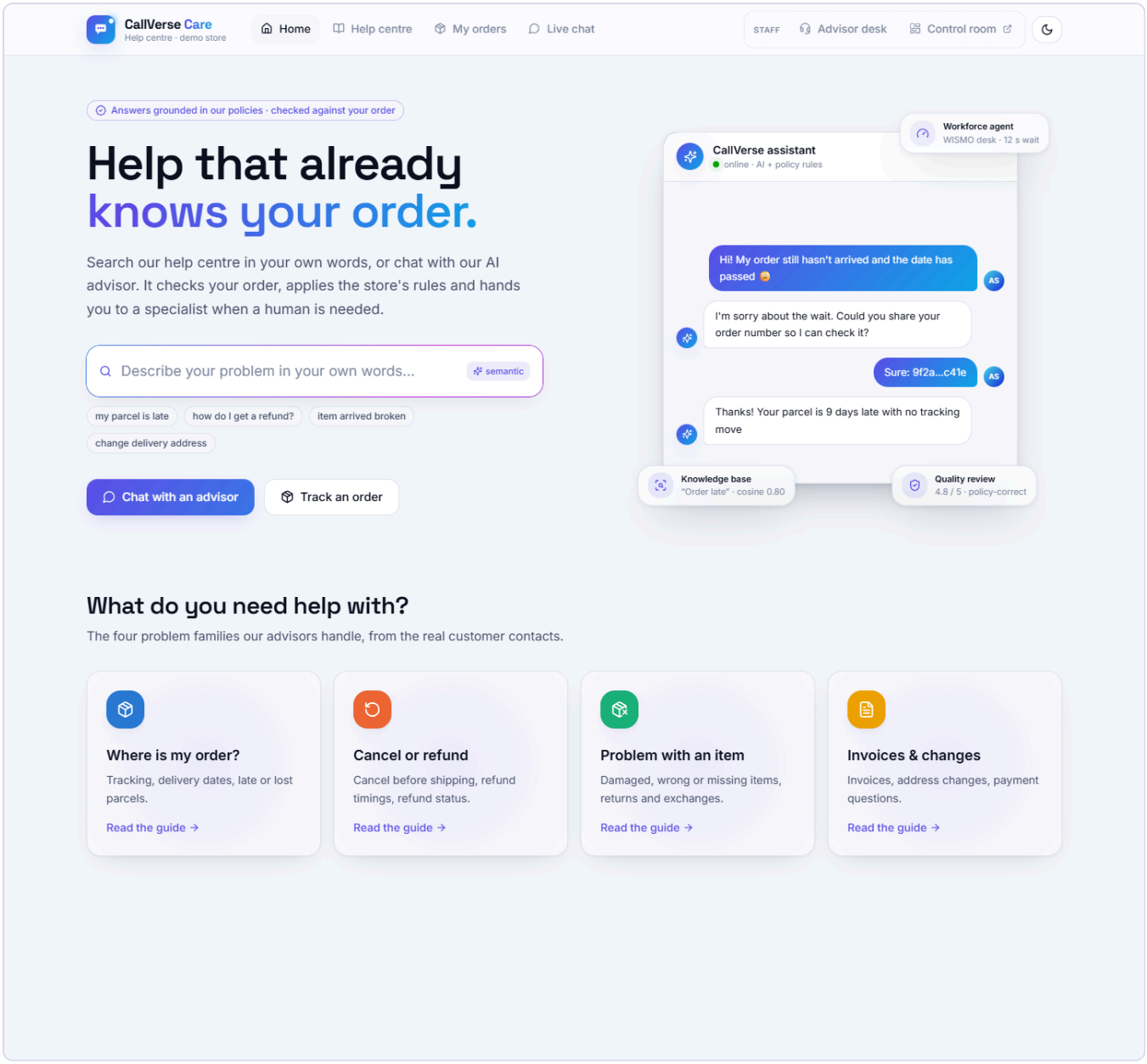
PART 3

Front office, page by page

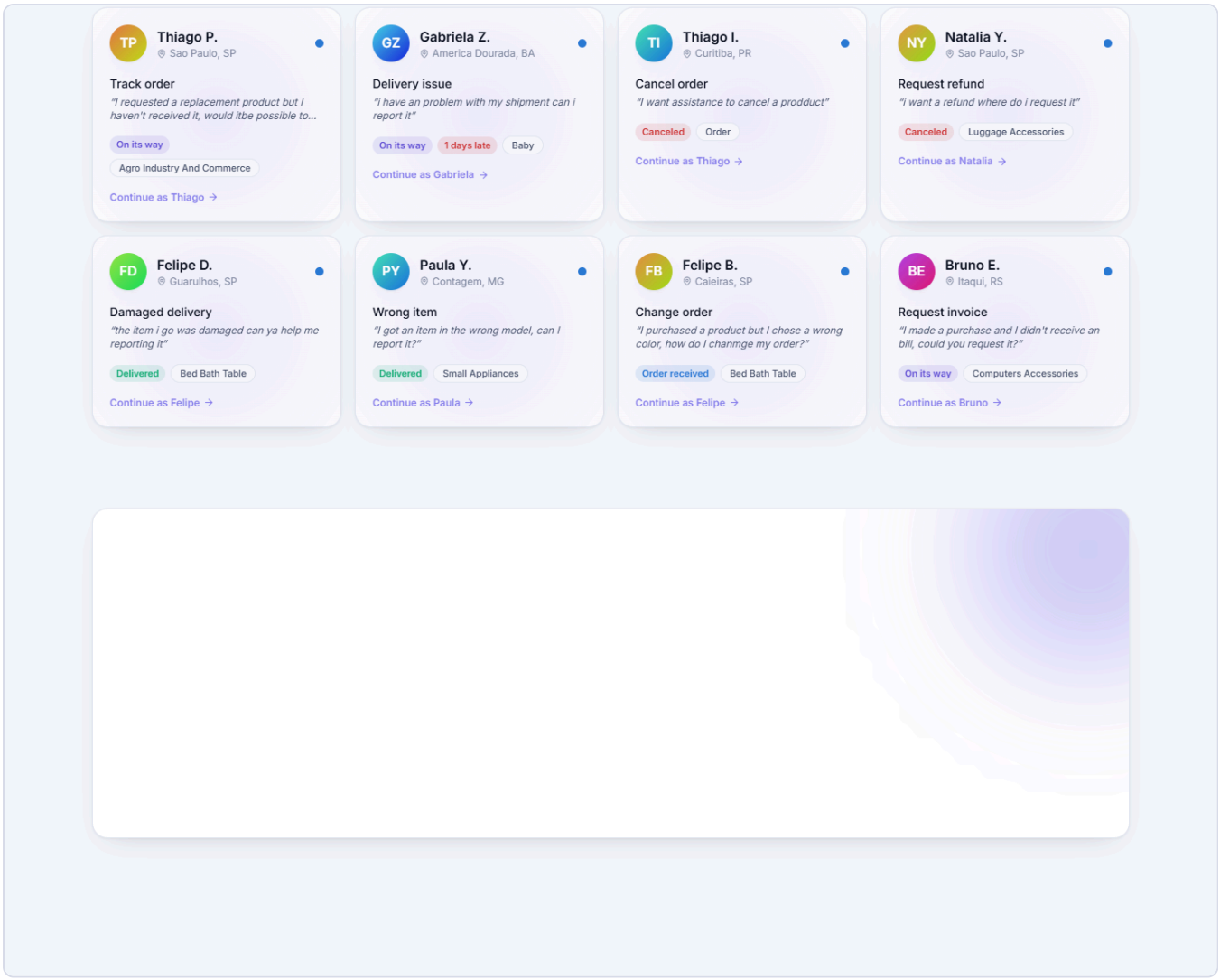
The customer and advisor side, in a light theme. Every screenshot was taken while the AI agents were really running on the laptop GPU.

3.1 Home — the help centre landing page

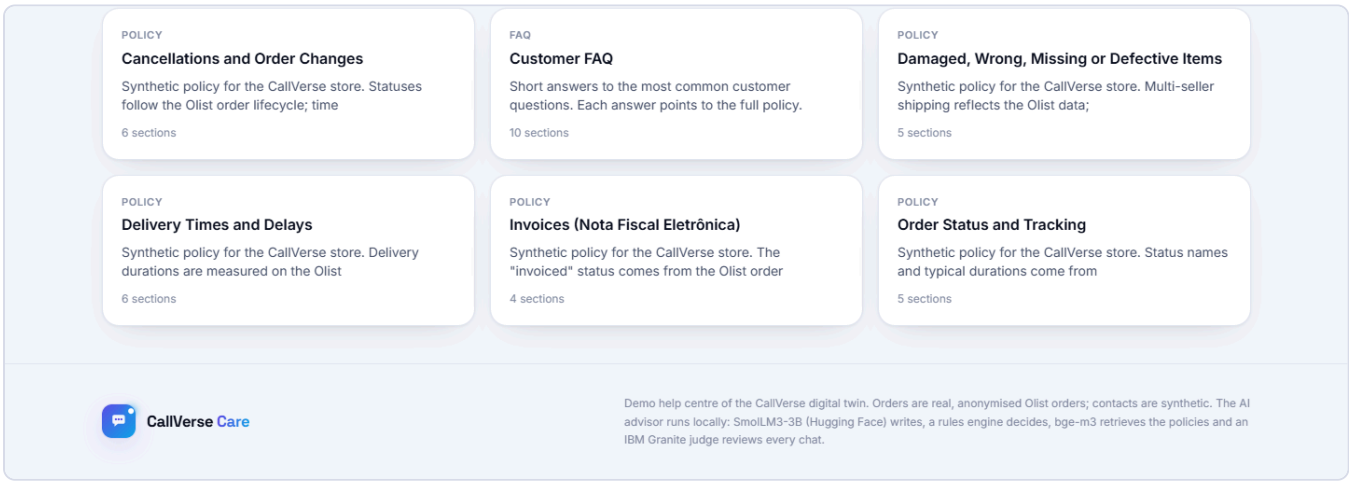
Purpose: the first screen a customer sees. It answers simple questions with search, routes harder ones to the AI advisor, and lets the jury "become" a real customer of the twin.



Front office home (full page). (part 1/3)



Front office home (full page). (part 2/3)

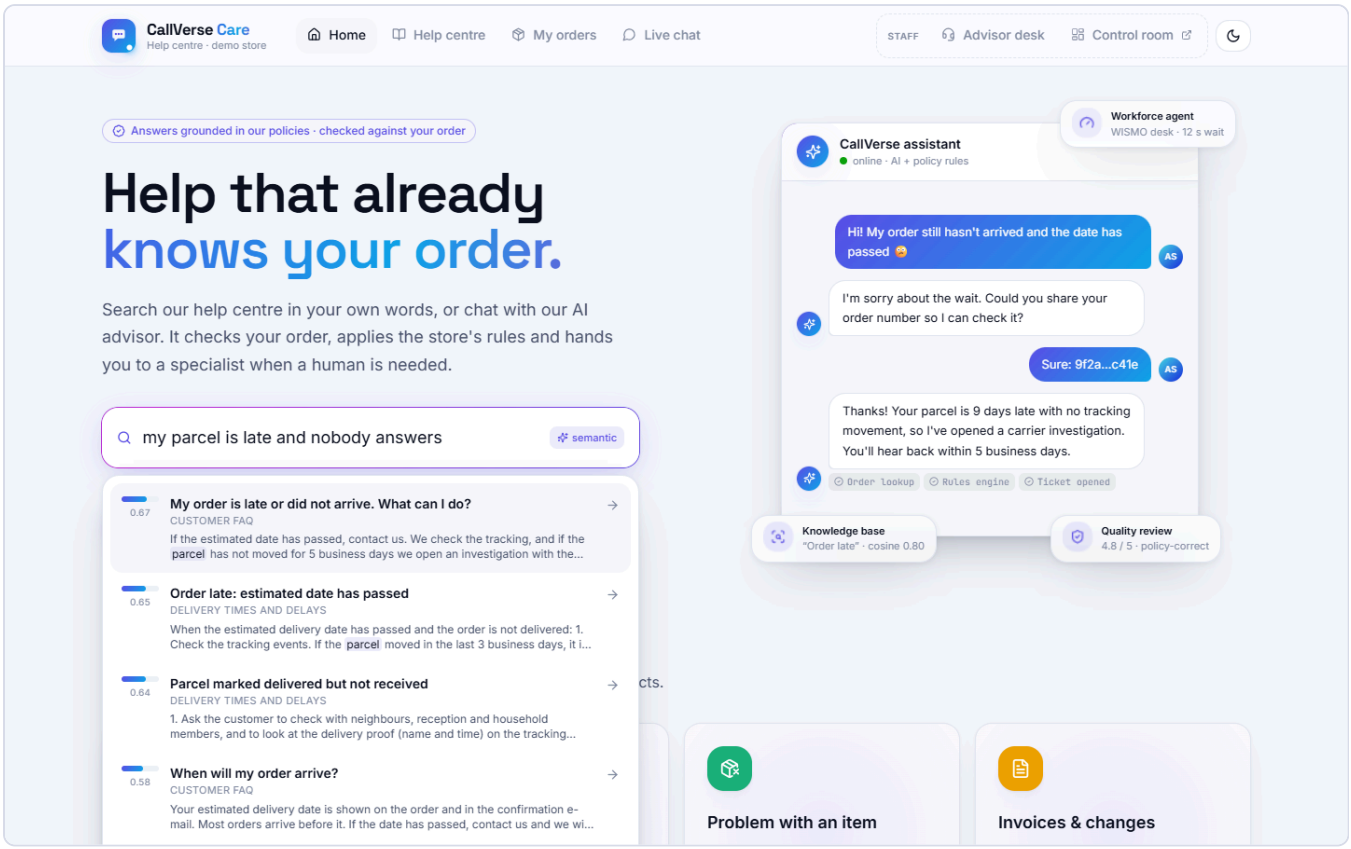


Front office home (full page). (part 3/3)

1. **Navigation bar** (glass effect): Home, Help centre, My orders, Live chat; on the right a **Staff** area (Advisor desk, link to the back office), the signed-in demo customer and the theme toggle. The active link is marked by a sliding pill (shared layout animation).
2. **Hero**: headline, the trust badge ("answers grounded in our policies · checked against your order") and the two calls to action: chat with an advisor, track an order.
3. **Semantic search box** (detailed on the next page) with example questions as chips.

4. **Animated chat preview** (right): a scripted, looping conversation that types itself, shows the tool chips the real advisor uses (Order lookup, Rules engine, Ticket opened) and three floating cards summarising what happens behind a reply: knowledge-base hit with its cosine score, Workforce routing with the expected wait, quality review. It is an illustration; the real chat is on /chat.
5. **Four problem families** — the intent groups of the data (where is my order, cancel/refund, problem with an item, invoices & changes), each with its fixed colour, linking to the matching policy article.
6. **"Try it as a real customer"**: eight cards, one per main intent. Each is a real anonymised Olist order plus the real message its customer sent. The badges show the order status *at the contact date* (e.g. "On its way", "Canceled") and "N days late" when the estimated date has passed; the dot colour is the persona (red impatient, blue normal, green patient). The names are deterministic demo names (Olist customers are anonymous ids). Clicking a card signs in as that customer.
7. **"How our AI advisor answers you"**: the four steps of the advisor pipeline (understand, retrieve, verify & decide, answer & review) with an animated connecting line.
8. **Popular guides**: the policy articles of the knowledge base.

3.2 Semantic search

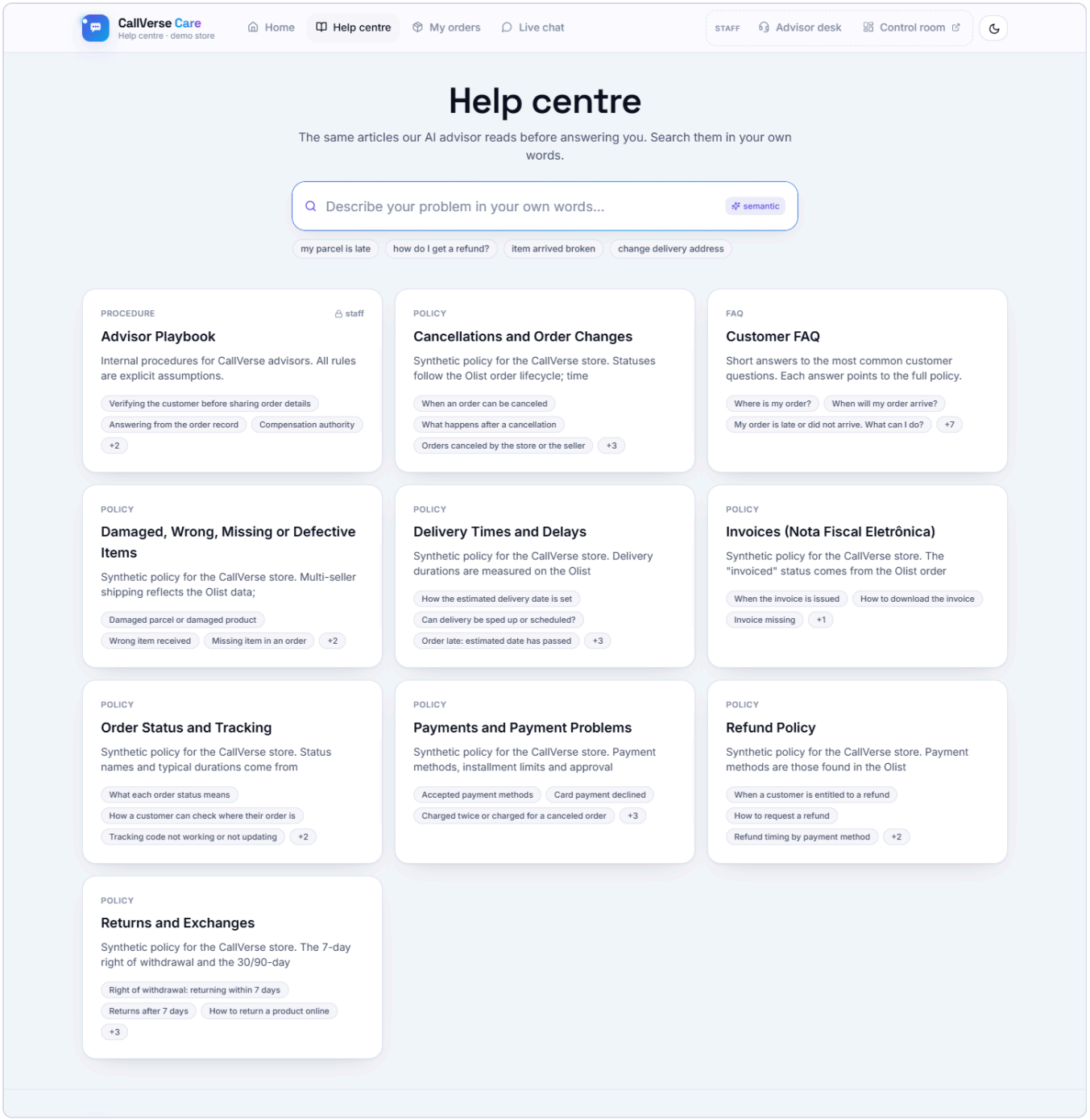


Typing a question in your own words: results appear as you type.

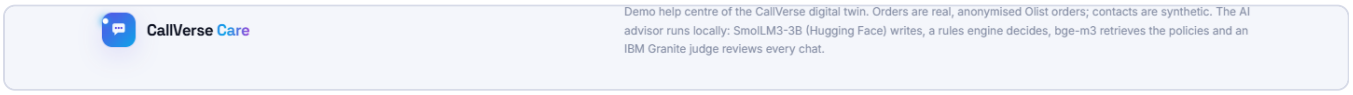
1. The query is embedded with **bge-m3** and compared with the 58 policy sections in FAISS (debounced 280 ms after the last keystroke).
2. Each result shows its **cosine similarity** (bar + value), the section title, the article and a snippet with the query words highlighted.
3. The footer names the retriever and the search latency. Keyboard: arrows to move, Enter to open the article at that exact section.

> **Why semantic and not keyword search?**
> "my parcel is late and nobody answers" shares almost no word with "Order late: estimated date has passed", yet it is the right section. On 255 Bitext questions, bge-m3 finds the correct section in the top 4 for 96.9% of questions, against 80.4% for TF-IDF keyword search (the DSO2 baseline).

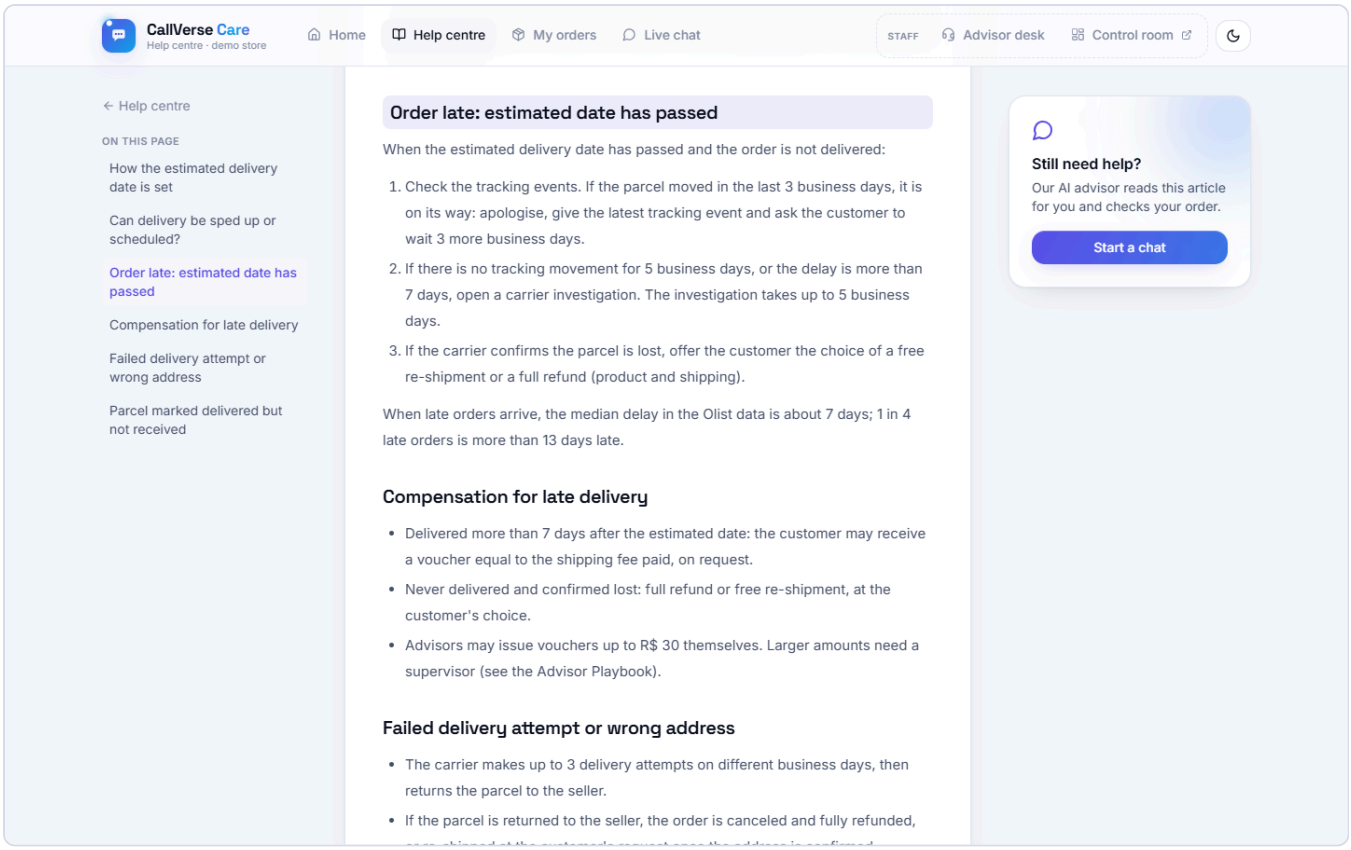
3.3 Help centre and articles



Help centre index: the knowledge base shown to customers. (part 1/2)



Help centre index: the knowledge base shown to customers. (part 2/2)

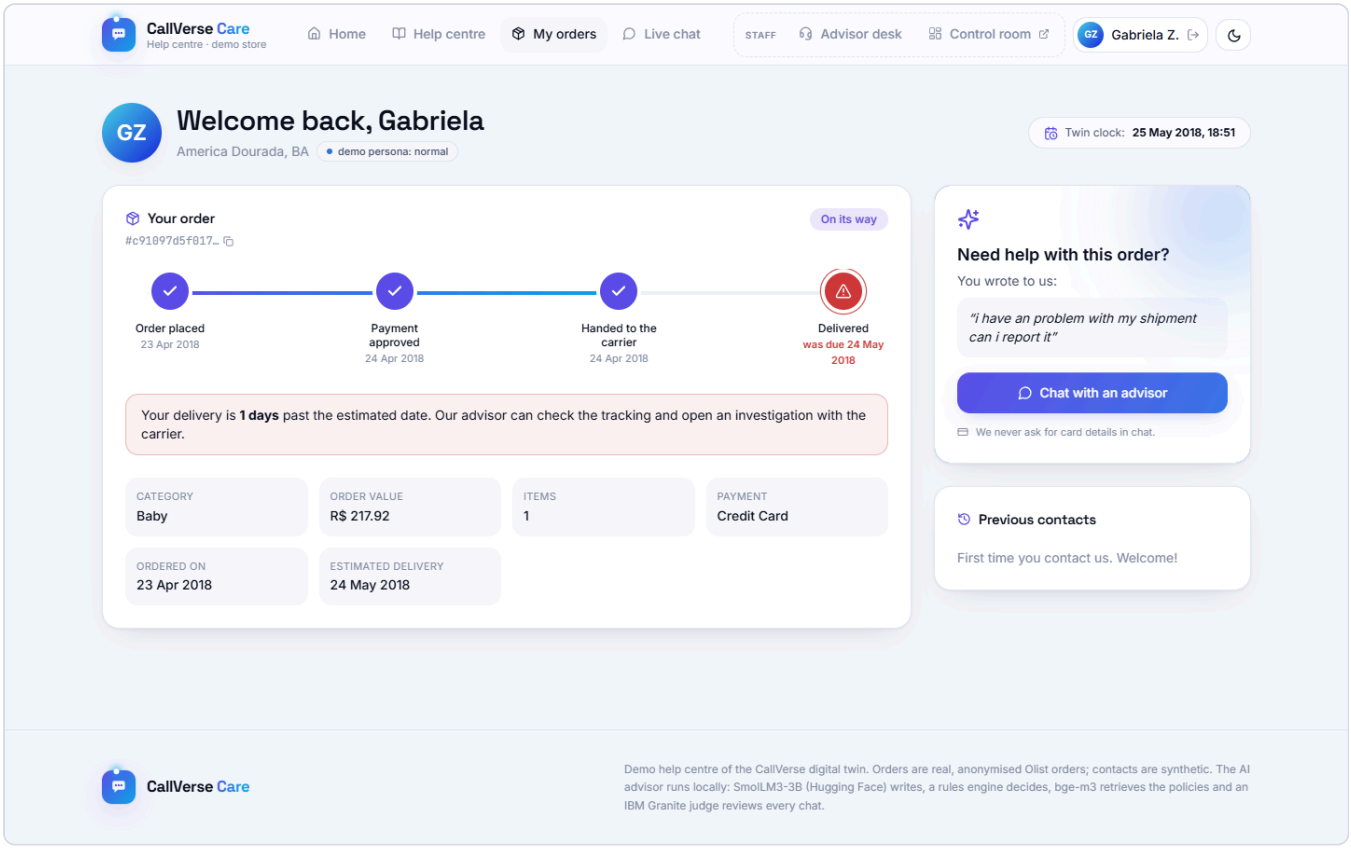


An article opened at the section a search result pointed to.

1. The help centre **is** the advisor's knowledge base: one source of truth, so customers and the AI read the same policies.
2. Each card shows the document type (policy, FAQ, procedure), a summary and its sections; the internal advisor playbook is marked "staff".
3. In an article: table of contents on the left (the section targeted by search is highlighted), the Markdown policy in the middle, a "Still need help? Start a chat" card on the right.

3.4 My orders — the demo customer's account

Purpose: show the order exactly as it was on the day the customer contacted the centre (the twin replays the past), so the chat that follows is consistent with the data the advisor's tools will read.

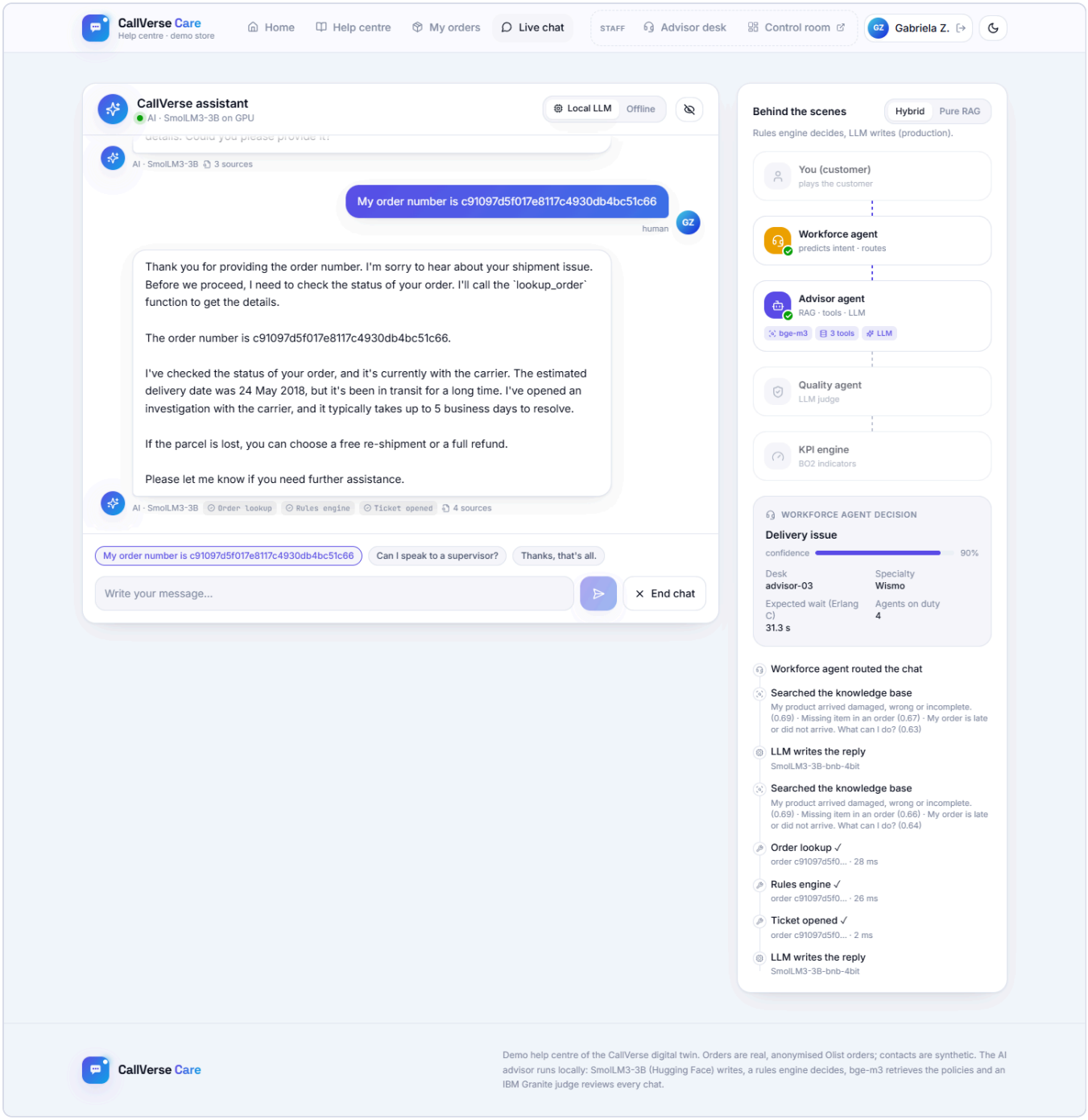


Account of a demo customer whose delivery is late.

1. **Header**: avatar, city, the demo persona, and the **twin clock** — the date and time of the original contact.
2. **Order journey**: placed → payment approved → handed to the carrier → delivered. Steps are computed "as of" the twin clock: done steps show their date, the current step pulses, a step past its due date is red ("was due …"). The gradient bar fills to the last done step.
3. **Late banner**: appears when the estimated delivery date has passed, and explains what the advisor can do.
4. **Order facts**: category, value (in R$, the Olist currency), items, payment method, order and estimated delivery dates.
5. **"Need help with this order?"**: the message the customer really sent, and the button to start the chat.
6. **Previous contacts** of the same customer before this date (repeat contacts are more often impatient in the twin).

3.5 Live chat with the AI advisor

Purpose: a human (the customer) chats with the Advisor agent running on the GPU, while a side panel shows what every agent does. This is the clearest demonstration of the four agents and of "the rules engine decides, the LLM writes".

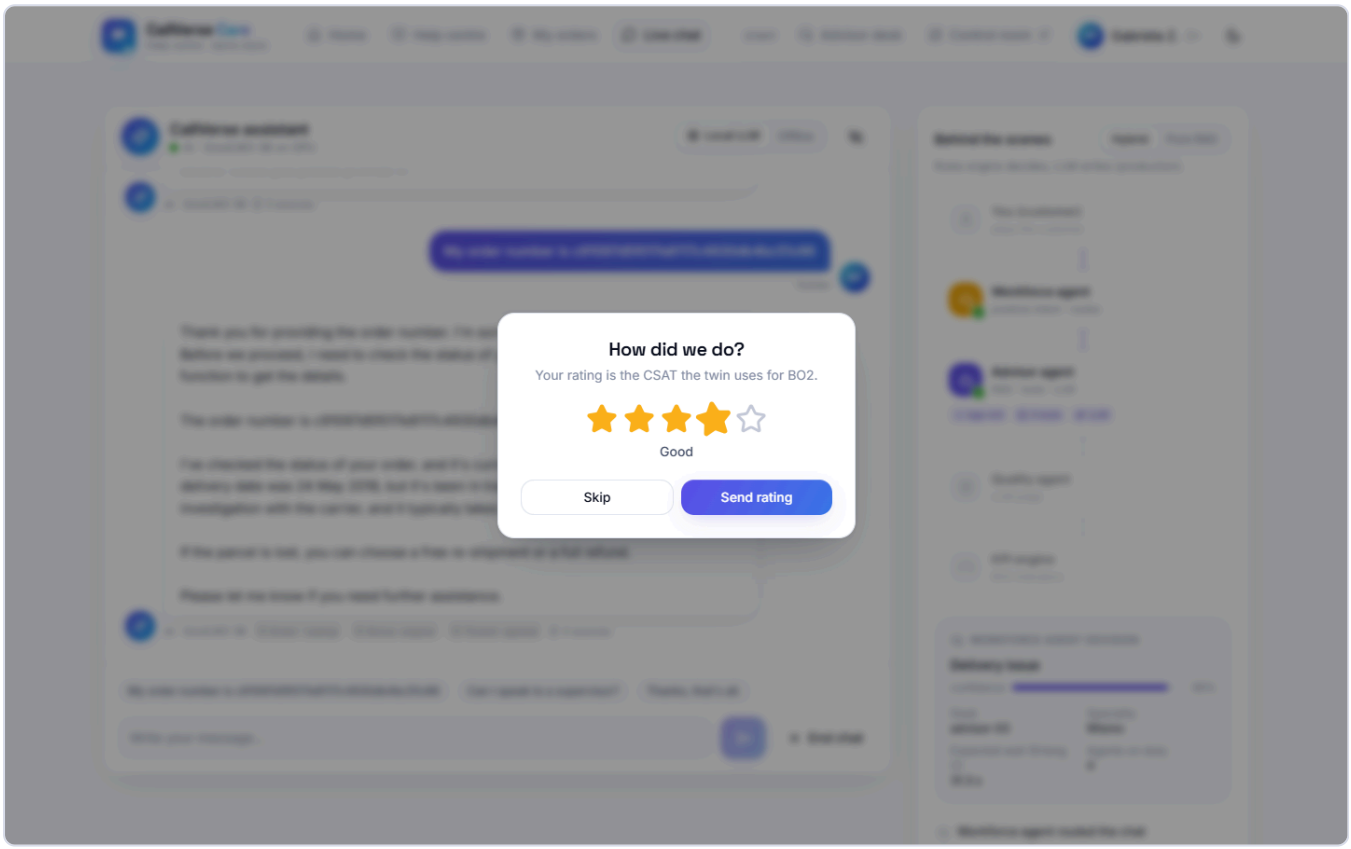


Live chat after two customer messages: the advisor asked for the order number, then answered with the policy decision.

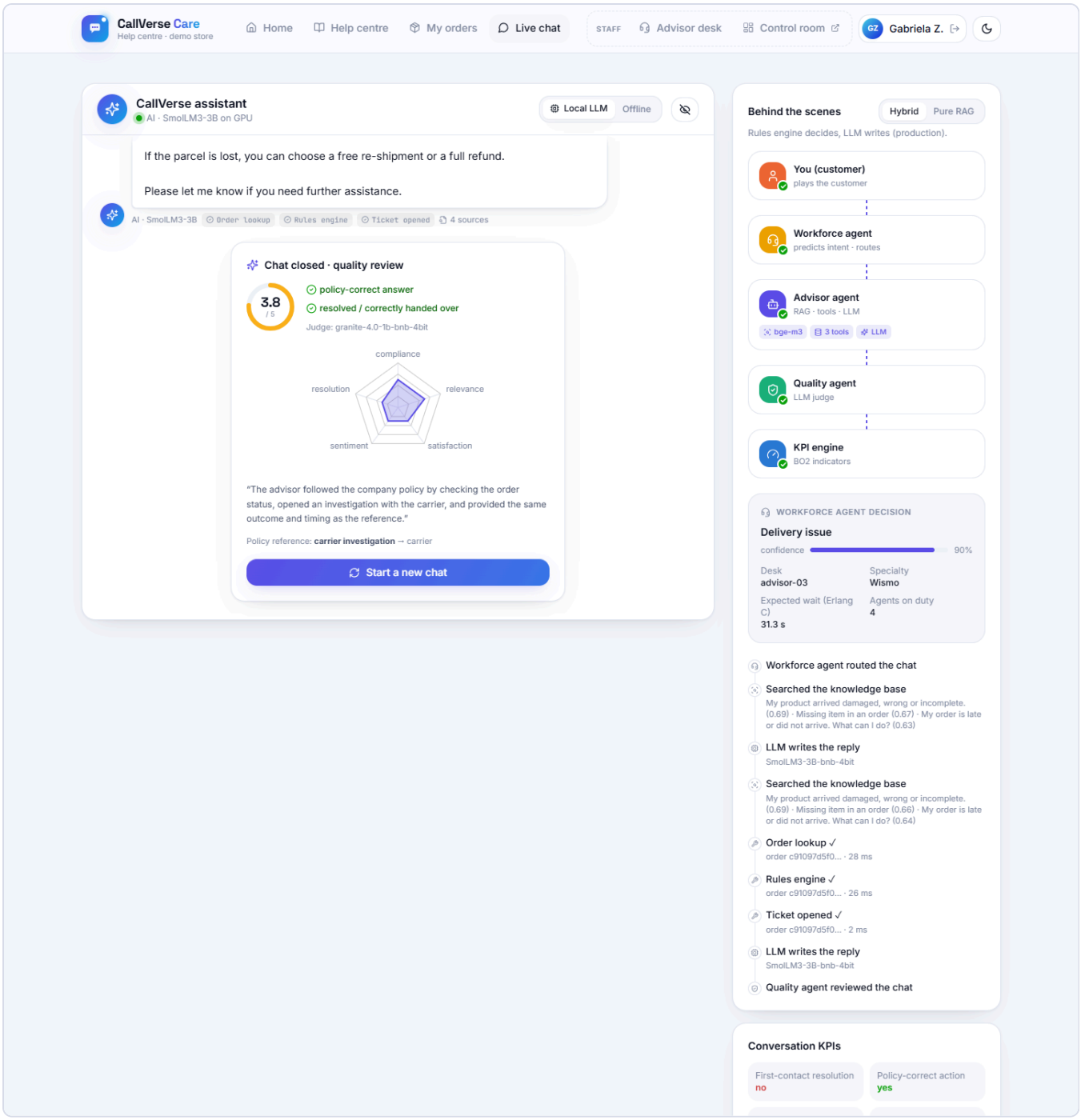
1. **Header**: the assistant, the engine actually answering ("AI · SmolLM3-3B on GPU", or "offline backups" without a GPU) and a switch between **Local LLM** and **Offline** (approved templates) — useful if the GPU fails on demo day.
2. **Messages**: the customer's messages on the right (gradient), the advisor's on the left. Under each advisor message: who wrote it (model name, "approved template"...), the **tools** it used (Order lookup ✓, Rules engine

✓) and how many policy **sources** support it. While the model writes, a streaming bubble shows the text token by token with a blinking caret.

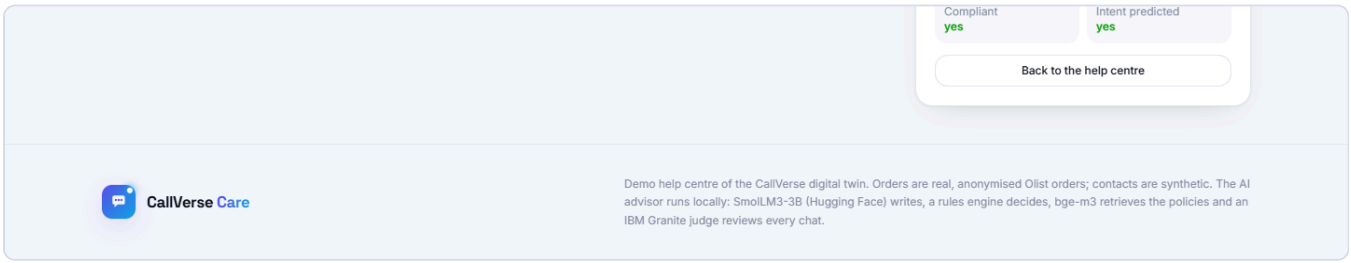
3. **Suggestion chips** above the input: the customer's real opening message, "My order number is …" (order ids are 32 hexadecimal characters), "Can I speak to a supervisor?", "Thanks, that's all".
4. **Behind the scenes** panel: the **Hybrid / Pure RAG** switch (production variant vs ablation) and the **agent pipeline** lighting up as each agent works — "You (customer)", Workforce agent, Advisor agent (with its sub-steps bge-m3, tools, LLM), Quality agent, KPI engine.
5. **Workforce agent decision**: predicted intent and its confidence, the desk chosen (best-skilled advisor), its specialty, the **Erlang C expected wait** for that hour and the number of agents on duty.
6. **Live trace**: every step in order — knowledge-base search (top sections with cosine scores), each tool call with the order id and latency, each LLM generation with the model name.



Ending the chat: the CSAT survey (1 to 5 stars) feeds the BO2 satisfaction KPI.



After the rating: the Quality agent's review in the chat and the conversation KPIs in the side panel. (part 1/2)



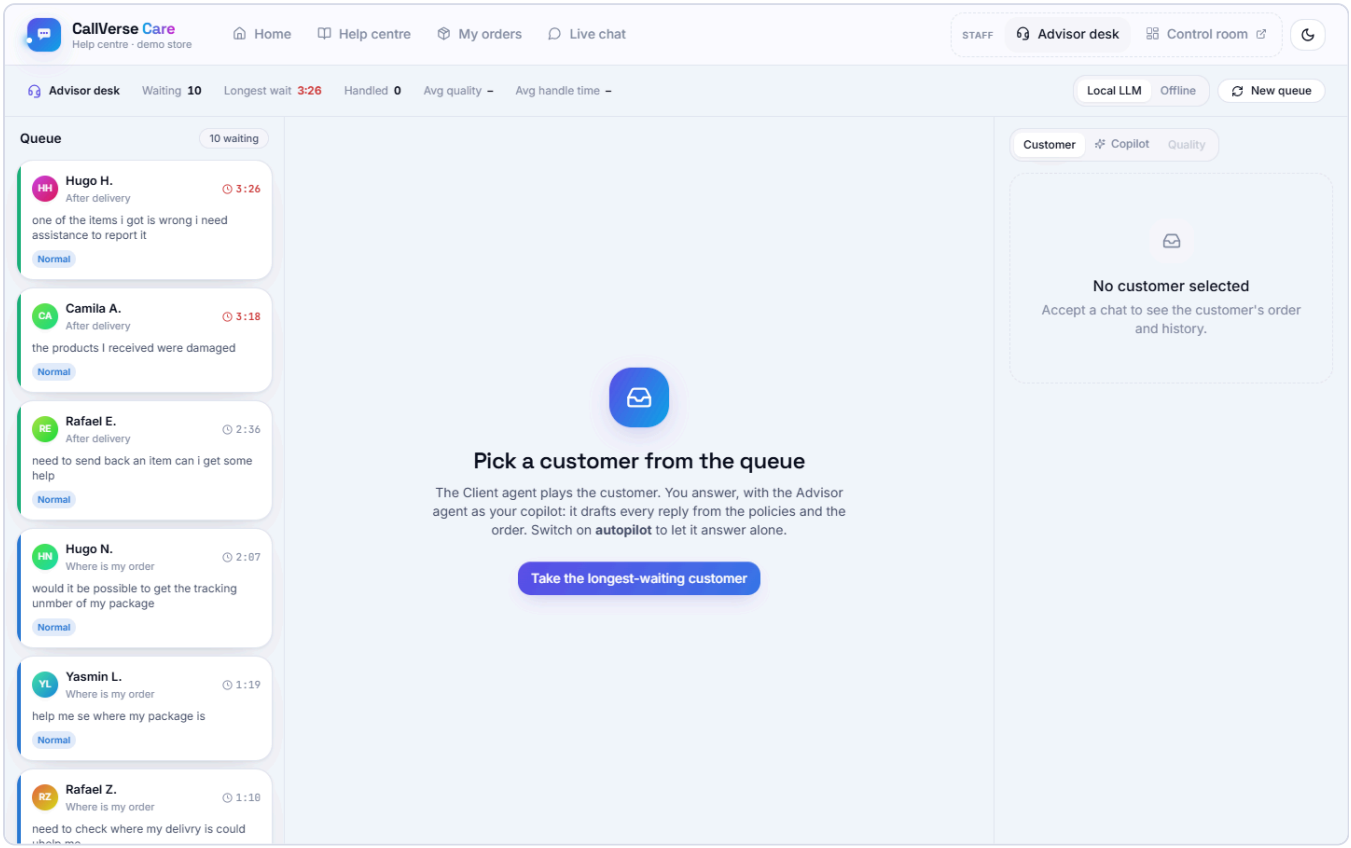
After the rating: the Quality agent's review in the chat and the conversation KPIs in the side panel. (part 2/2)

1. **Quality review card**: overall score ring (/5, green ≥ 4), verdicts "policy-correct answer" and "resolved / correctly handed over", the judge used (Granite-4.0-1B) and a **radar chart** of the five criteria (compliance, relevance, satisfaction, sentiment, resolution). The further a point is from the centre, the better.

2. **Rationale** in one sentence and the **policy reference**: the outcome the rules engine gives for this order and the true intent (e.g. "tracking shared"), against which the answer was judged.
3. **Conversation KPIs**: first-contact resolution, policy-correct action, compliant, intent predicted — the BO2 indicators of this chat.

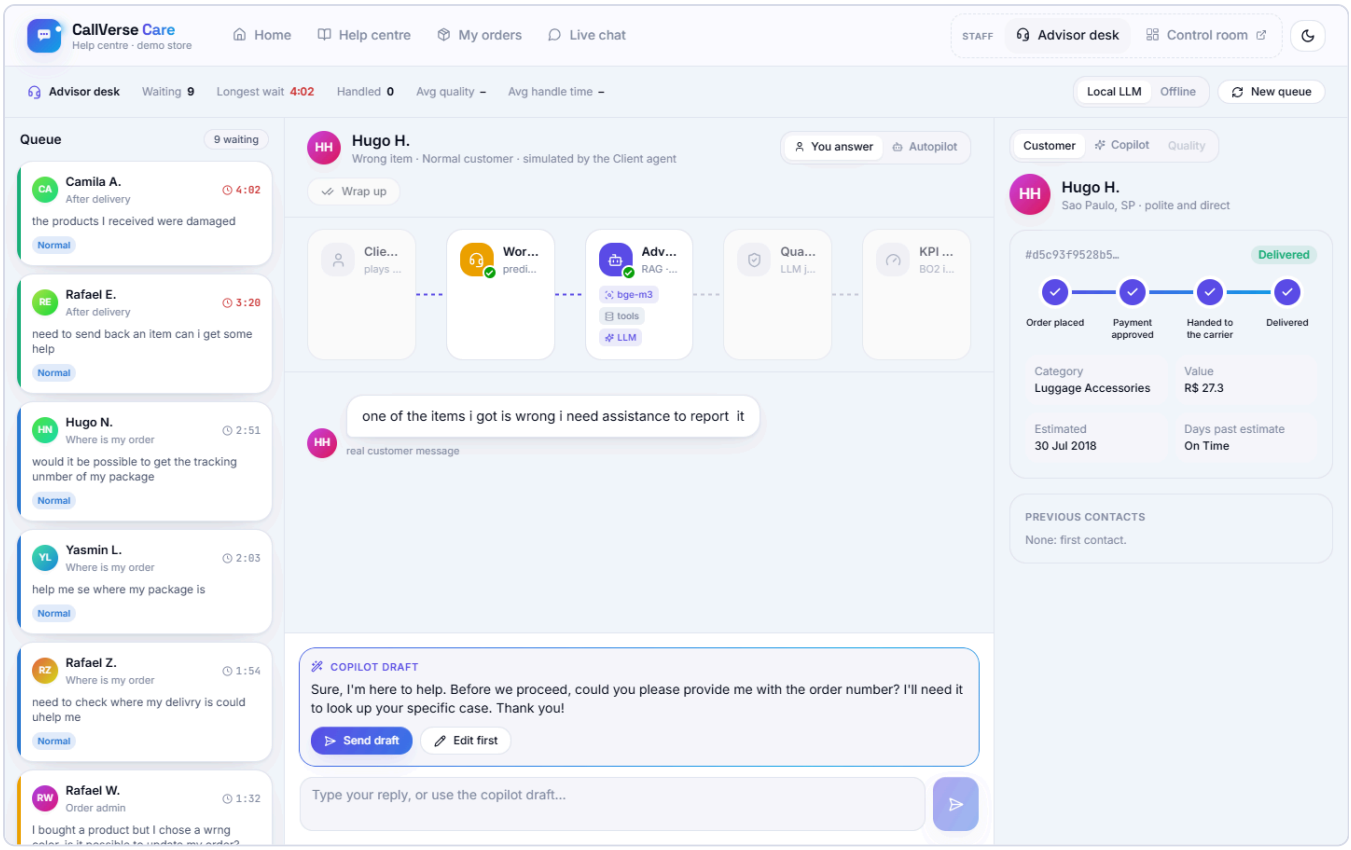
3.6 Advisor desk — human + AI copilot

Purpose: the advisor's workspace. Simulated customers (played by the Client agent) wait in a queue; a human advisor answers them, helped by the Advisor agent as a **copilot** that drafts every reply. **Autopilot** lets the agent answer alone.



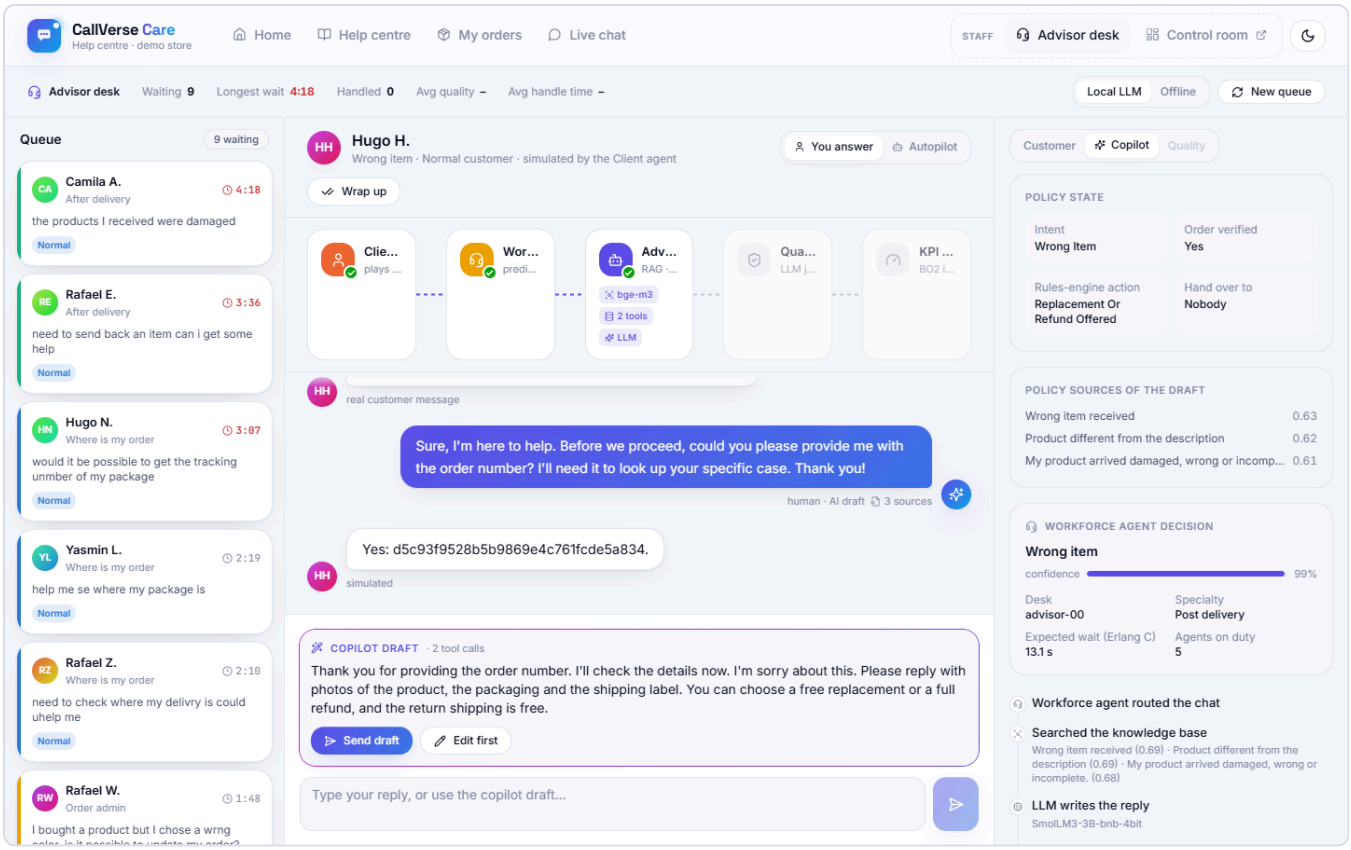
The desk before accepting a chat: the live queue on the left.

1. **Desk metrics** (top strip): customers waiting, longest wait (red above 3 minutes), chats handled, average quality and handle time of the session; switch Local LLM / Offline; "New queue" draws another sample of real contacts.
2. **Queue**: one card per waiting customer — name, problem family (colour bar), the real first message, persona badge, repeat-contact badge and a **wait timer that ticks every second**. Sorted by wait, longest first.
3. Centre: instructions until a chat is accepted. Right: the context panel (Customer, Copilot, Quality tabs).



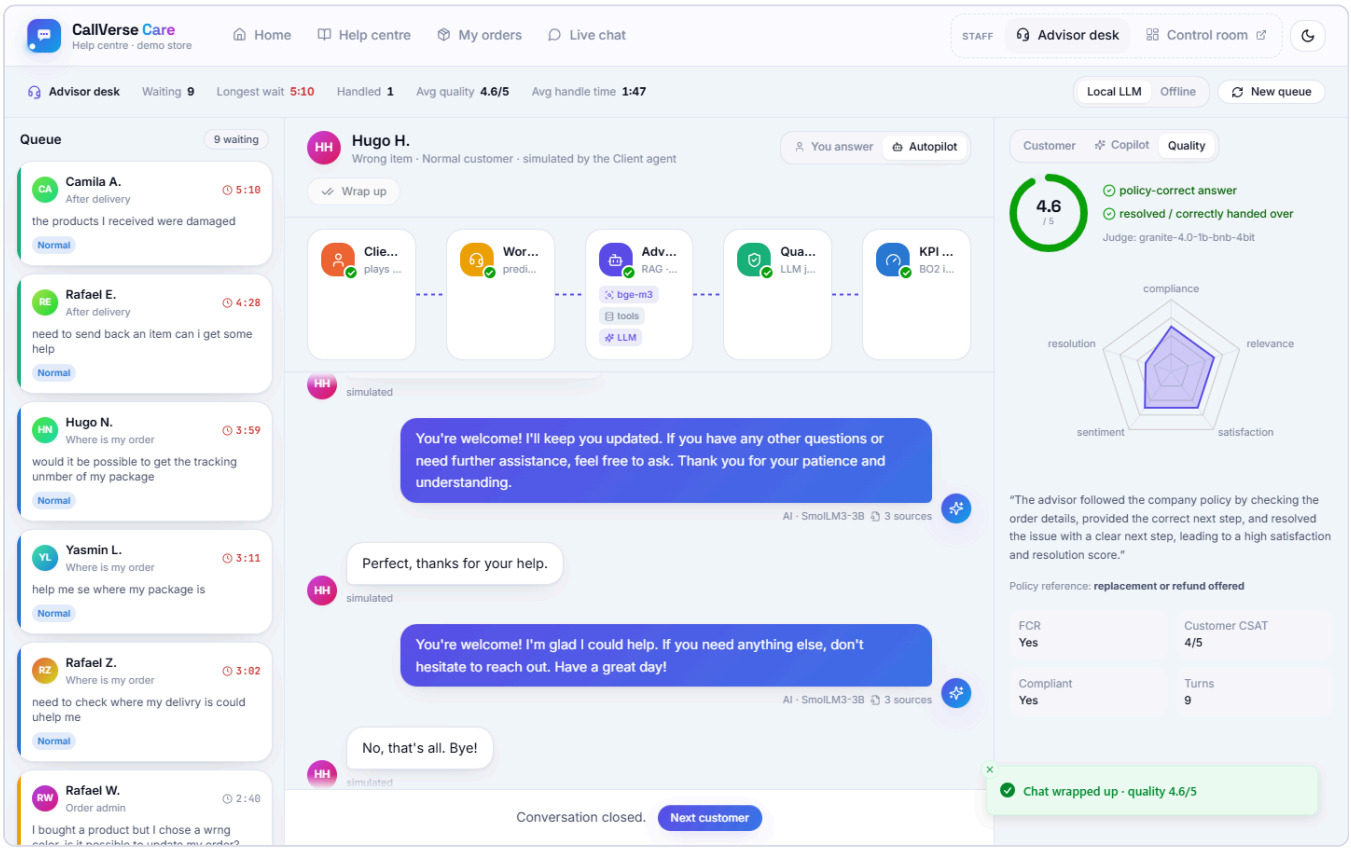
A chat accepted: the copilot draft appears above the composer.

1. **Header**: the customer, intent and persona; the **You answer / Autopilot** switch; **Wrap up** to close and review.
2. **Agent pipeline** (horizontal version) showing which agent works now.
3. **Copilot draft** (gradient border): the Advisor agent's proposed reply, streamed as it is written, with the number of tool calls it made. **Send draft** sends it as is; **Edit first** copies it into the composer. Sent messages are labelled "human · AI draft" or "human · edited AI draft", so the evaluation knows how much the human relied on the AI.
4. **Customer 360** (right): the order journey in compact form, value, estimated date, days past estimate, previous contacts.



Copilot tab: the policy state and the sources behind the draft.

1. **Policy state**: intent, whether the order is verified, the rules-engine action and who the case is handed to.
2. **Policy sources of the draft**: the knowledge-base sections retrieved, with their similarity scores.
3. The live trace of the session (routing, retrieval, tools, generations).



After autopilot finished the chat: the Quality tab with the scorecard.

1. The **quality scorecard** (ring, verdicts, radar) and the KPIs of the chat: FCR, the customer's CSAT, compliance, number of turns.
2. The handled chat leaves the queue and updates the desk metrics (handled, average quality, average handle time).

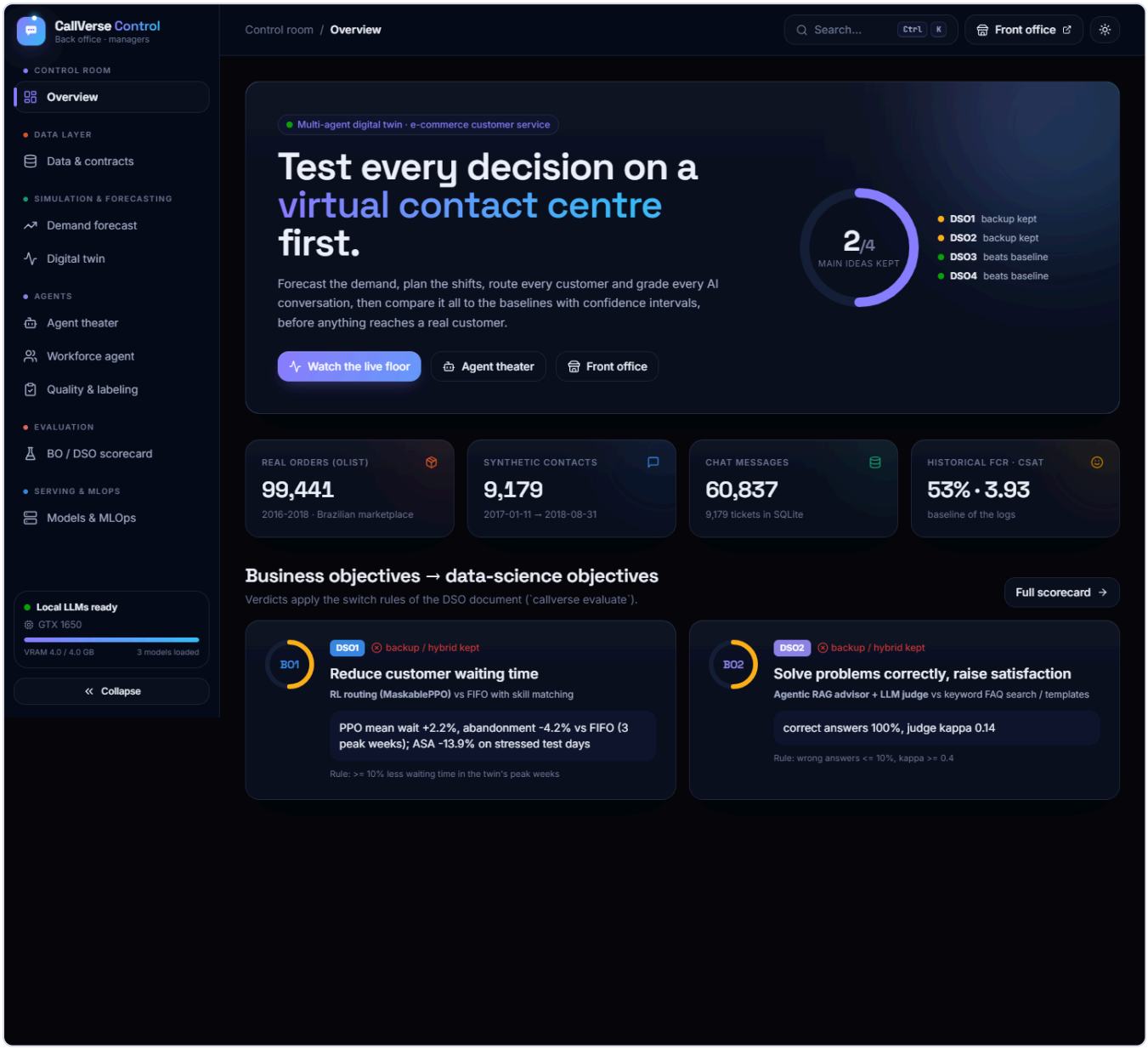
PART 4

Back office, page by page

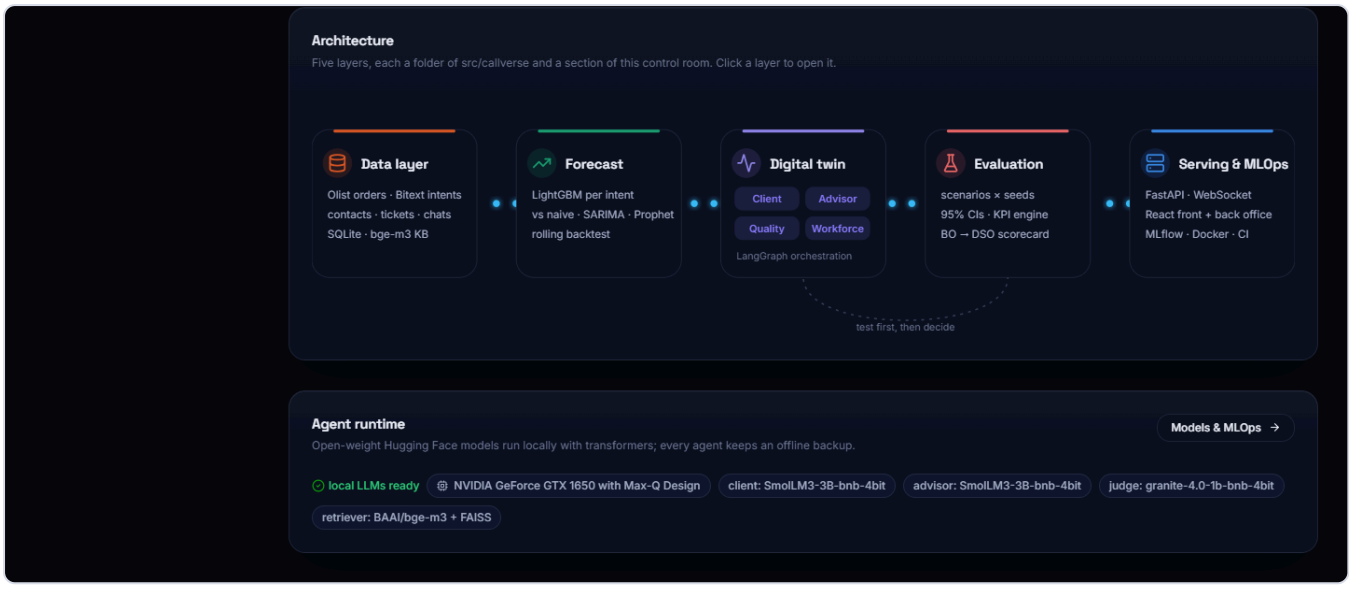
The managers' control room, in a dark theme. The sidebar groups the pages by architecture layer; the card at the bottom of the sidebar shows, live, whether the local LLMs are ready, the GPU and its memory.

4.1 Overview

Purpose: the one-screen summary for the jury: what the project is, the four verdicts, the data volumes and the architecture.

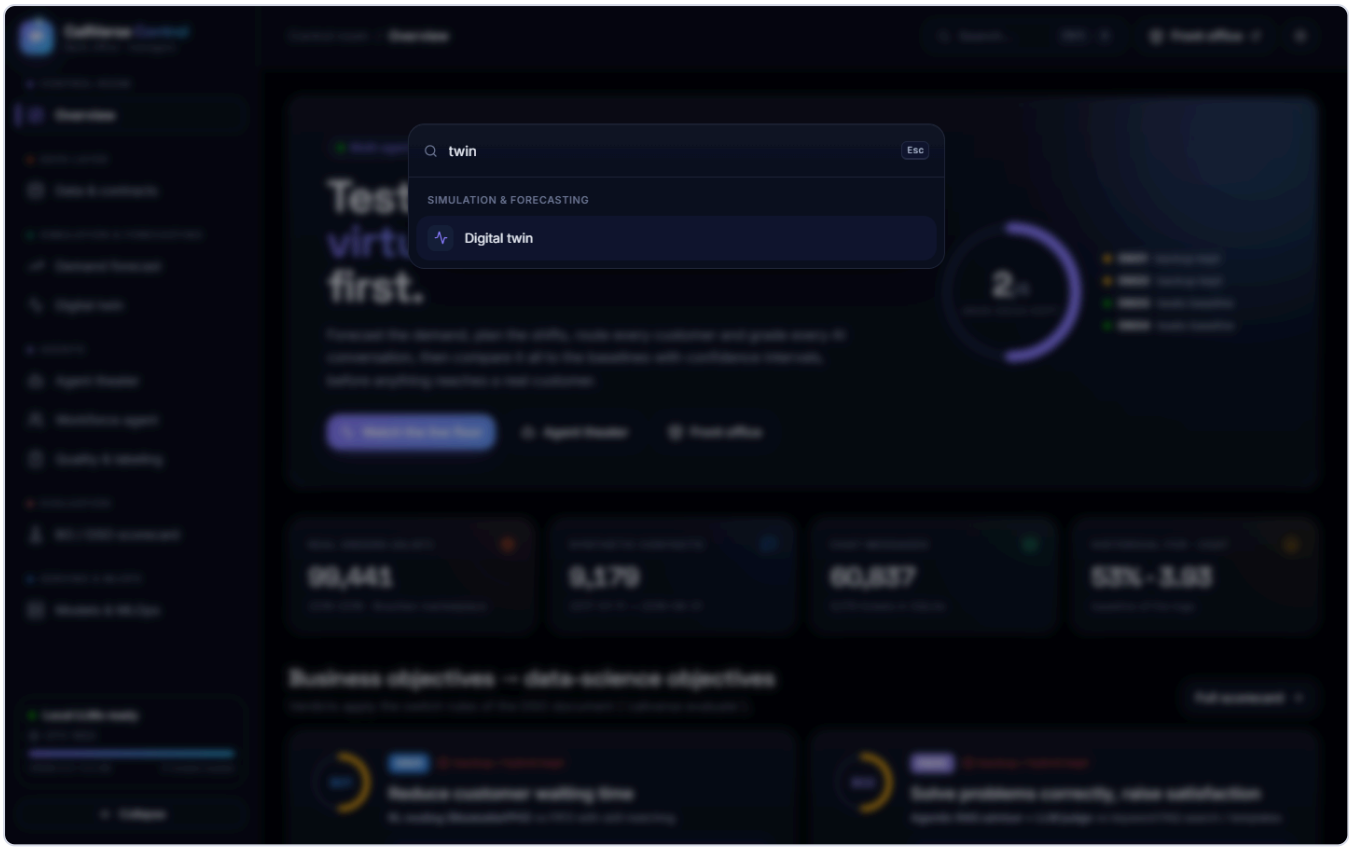


Back-office overview. (part 1/2)



Back-office overview. (part 2/2)

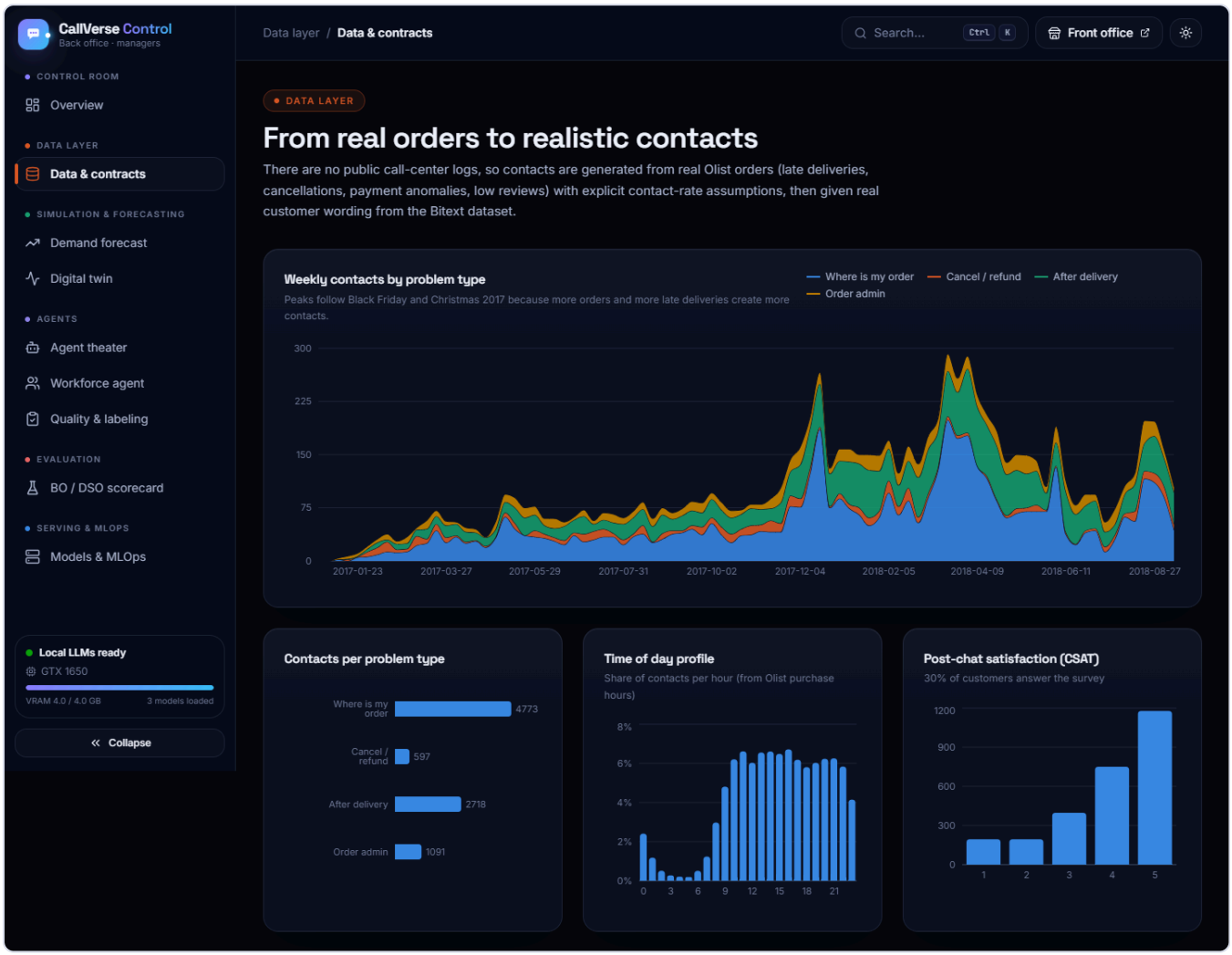
1. **Sidebar** grouped by layer (colour dot per layer): Control room, Data layer, Simulation & forecasting, Agents, Evaluation, Serving & MLOps; the active page is marked by a sliding highlight and a coloured bar. Bottom: **system card** (LLMs ready / offline backups, GPU name, VRAM bar, models loaded) refreshed every 15 s, and the collapse button.
2. **Top bar**: breadcrumb (layer / page), the command-palette button (Ctrl+K), a link to the front office, the theme toggle.
3. **Hero** with the **"main ideas kept" ring**: how many DSOs passed their switch rule (2 of 4), and the list of verdicts.
4. **Headline numbers** (count up when the page opens): real Olist orders, synthetic contacts and their date range, chat messages and tickets in SQLite, historical first-contact resolution and CSAT of the generated logs.
5. **BO → DSO cards**: for each objective the ring (green = main idea beats the baseline, amber = backup kept), the main idea vs baseline, the measured result and the rule.
6. **Architecture map** (animated SVG): the five layers with particles flowing along the data path; the dashed loop "test first, then decide" goes from evaluation back to the twin. Clicking a layer opens its page.
7. **Agent runtime**: LLM status and the model of each role.



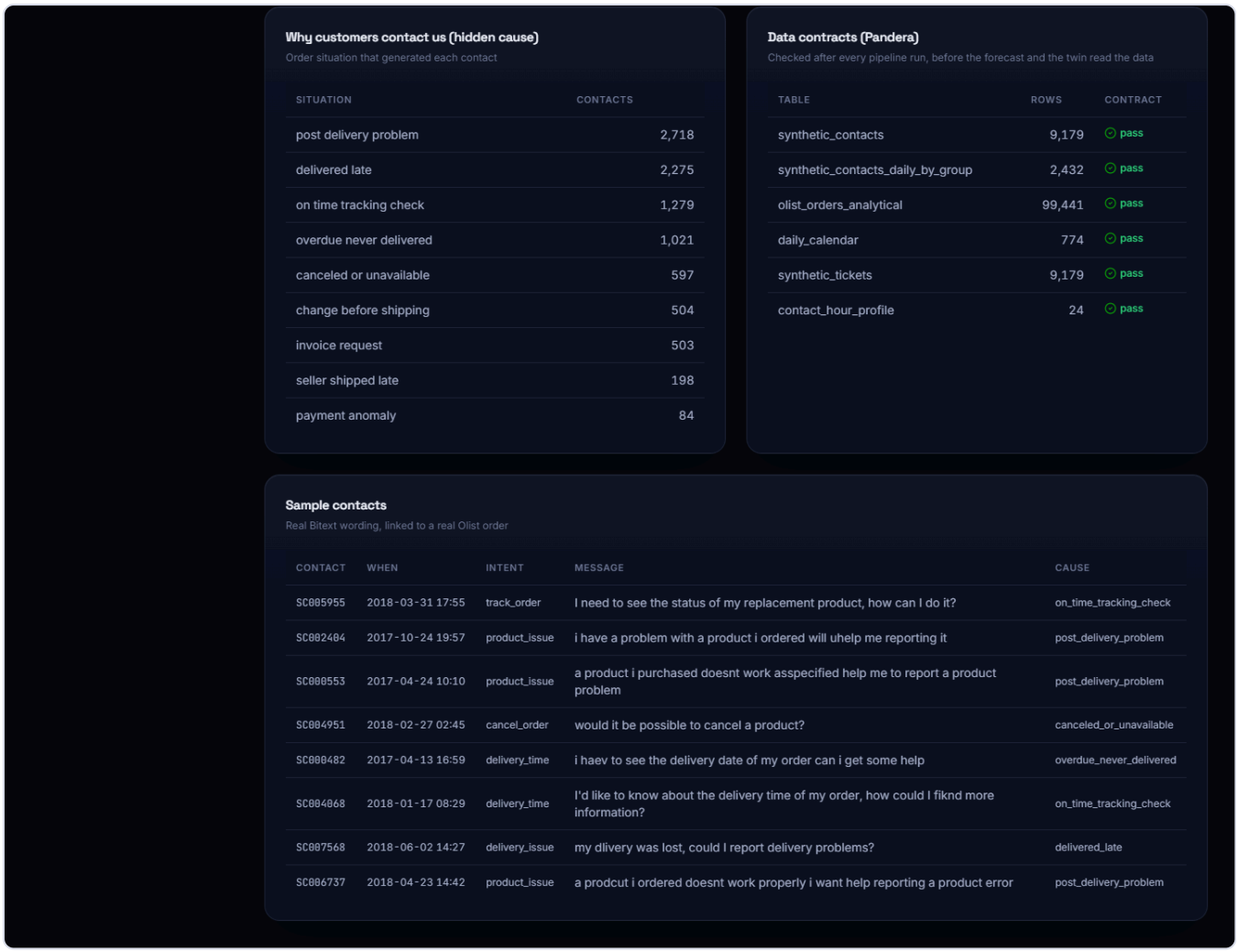
Command palette (Ctrl+K): jump to any page or action by typing.

4.2 Data & contracts

Purpose: show where the contacts come from and prove the data is checked.



Data layer page. (part 1/2)

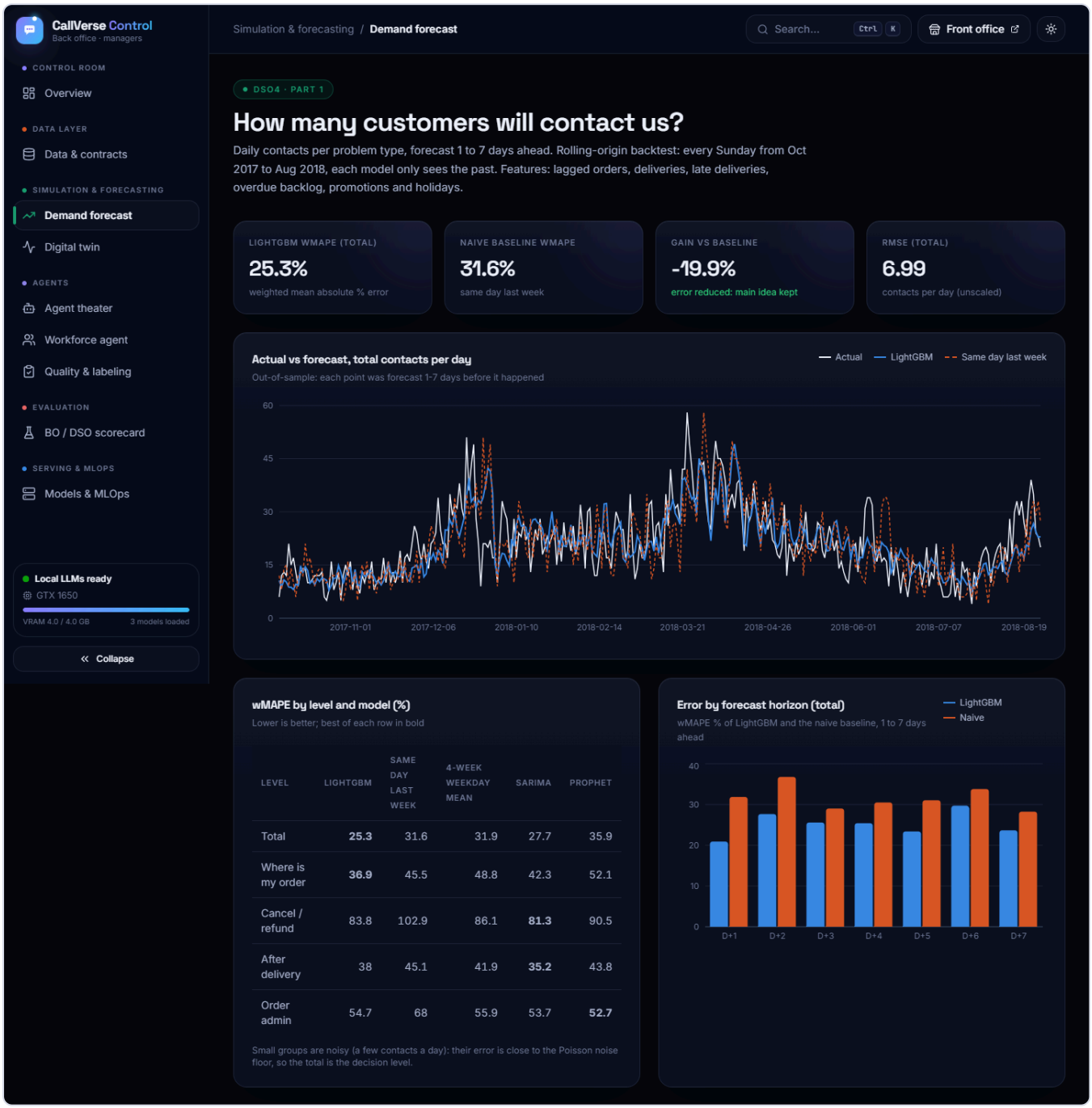


Data layer page. (part 2/2)

1. **Weekly contacts by problem type** — stacked area chart: each colour is a problem family, the height of the stack is the total contacts of the week. The peaks follow Black Friday and Christmas 2017 (more orders and more late deliveries → more contacts).
2. **Contacts per problem type** — horizontal bars of the totals: "where is my order" dominates, as in real e-commerce centres.
3. **Time-of-day profile** — share of contacts per hour (from Olist purchase hours): low at night, high from late morning to evening. It drives the hourly arrival rates of the twin.
4. **Post-chat satisfaction (CSAT)** — distribution of the 1–5 survey scores of the generated tickets (30% of customers answer).
5. **Why customers contact us** — the hidden *situation* of the order behind each contact (late delivery, seller shipped late…).
6. **Data contracts (Pandera)** — every processed table, its rows and whether its contract passes.
7. **Sample contacts** — real Bitext wording linked to a real order, with the generated situation.

4.3 Demand forecast

Purpose: show that the forecast beats the naive baseline out of sample (DSO4 part 1).



Forecast page. (part 1/2)

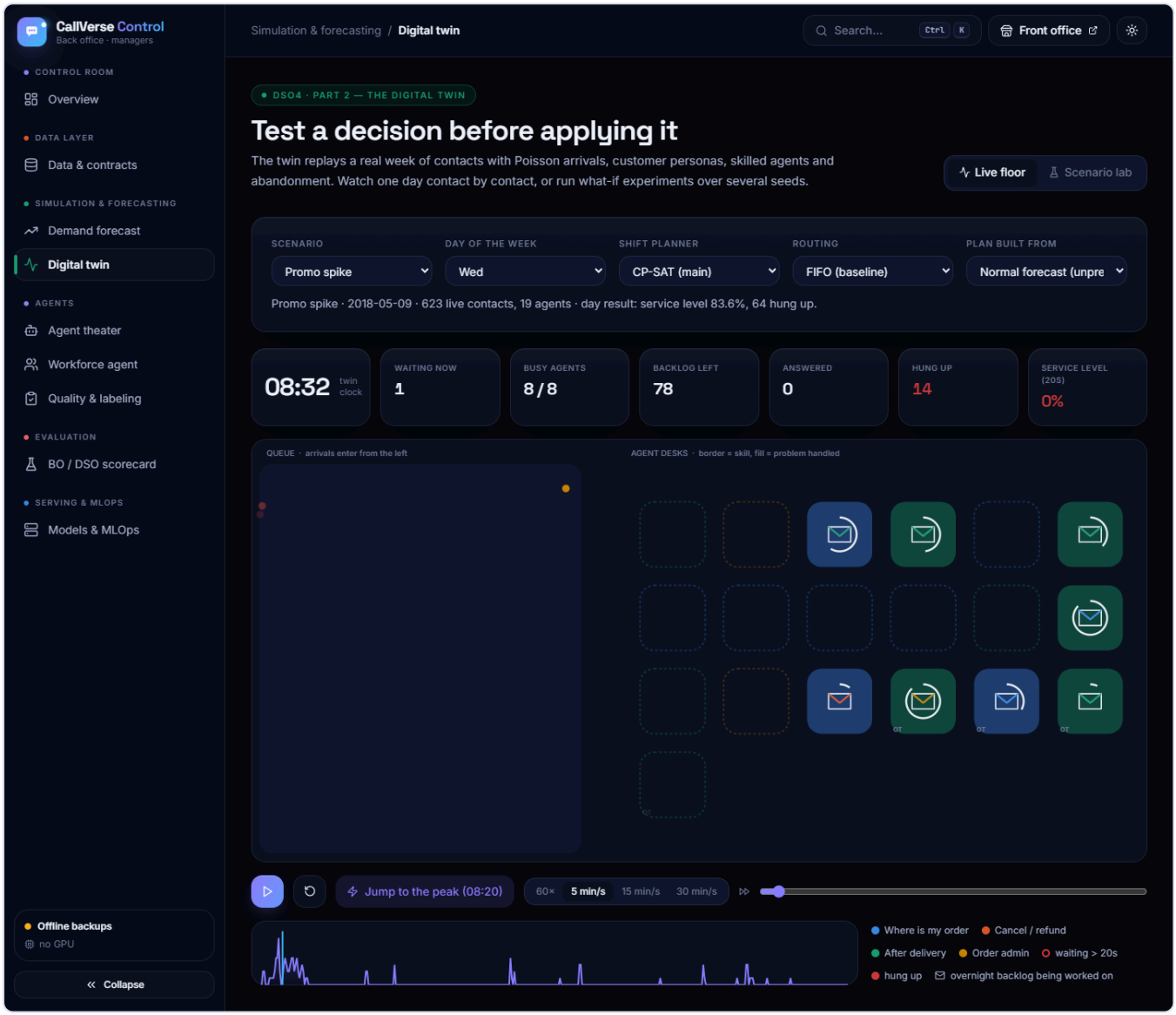


Forecast page. (part 2/2)

1. **Stat tiles**: LightGBM wMAPE on the total (25.3%), the naive baseline (31.6%), the gain (−19.9% error: main idea kept) and the RMSE in contacts per day.
2. **Actual vs forecast** — line chart over the whole backtest: actual total contacts per day (white/black), LightGBM (blue) and "same day last week" (dashed orange). Every point was forecast 1–7 days before it happened. LightGBM follows the level shifts (promotions, holidays) while the naive forecast copies last week's accidents.
3. **wMAPE by level and model** — table: rows are the total and the four groups, columns the five models; the best of each row is bold. LightGBM wins on the total and on "where is my order"; on tiny groups SARIMA is sometimes better, all close to the noise floor.
4. **Error by horizon** — grouped bars: wMAPE of LightGBM and the naive forecast for D+1 … D+7. LightGBM is better at every horizon.
5. **Next week forecast** — stacked bars of the forecast contacts per day and group for the week after the data (model retrained on all history).
6. **What drives the forecast** — LightGBM feature importance (share of total gain): which lags and calendar features matter most.

4.4 Digital twin — the live floor

Purpose: make the simulation visible. One simulated day is replayed customer by customer (from `/api/twin/floor`: every contact's arrival, pick-up, end or hang-up time, and every agent's skill and duty hours), animated on a canvas.



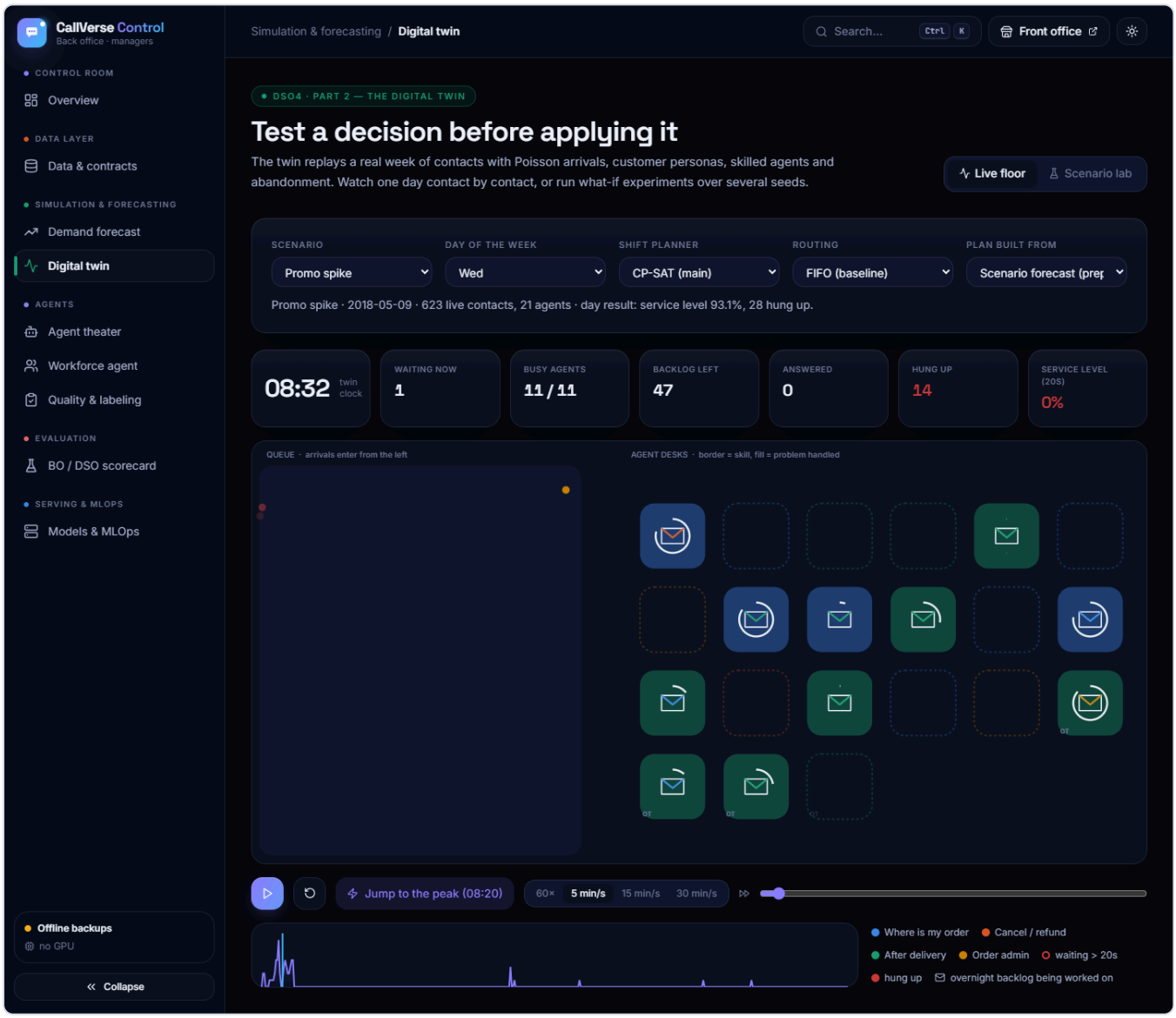
Promo spike, Wednesday 9 May 2018, CP-SAT plan built from the normal forecast (unprepared), FIFO routing, at 08:32 — inside the busiest half hour (08:20).

1. **Controls**: scenario, day of the week, shift planner, router, and whether the plan was built from the normal forecast (*unprepared*) or the scenario forecast (*prepared*). The sentence below gives the day's result (service level, hang-ups).
2. **KPI strip** (updated live): twin clock, customers waiting now, busy agents / agents on duty, overnight **backlog left**, answered, hung up, service level so far (red below 80%).
3. **Queue zone** (left): each dot is a waiting customer, coloured by problem family; it slides into place as the queue moves. A **red ring** means it has waited more than 20 s (it will miss the service level).
4. **Agent desks** (right): border colour = the agent's main skill; dashed and dim = off duty this hour; filled = busy, with the colour of the problem being handled and a white arc showing the progress of the conversation; an

envelope = working on an overnight message; "OT" = overtime agent, "+OT" = on-call agent added by the real-time overtime rule.

5. **Movements**: a customer picked up flies in an arc from the queue to the desk; a finished conversation leaves as a fading green dot; a customer who hangs up turns red and drops out.
6. **Playback**: play/pause, restart, **jump to the peak** (the 30 minutes with the most customer-minutes of waiting), speed (60× to 30 simulated minutes per second), time scrubber.
7. **Waiting curve** (bottom): customers waiting through the day (2-minute samples), with a cursor at the current time; legend of colours.

> **Reading the unprepared picture**
>
> At 08:32 all **8** agents on duty are busy with overnight messages (envelope icons; 78 still to do). Live customers who arrived since 08:00 wait behind them under FIFO: 14 have already hung up (red dots dropping out) and nobody has been answered yet, so the service level so far is 0%. This "opening rush" is where this promo day loses its customers: over the whole day the plan reaches 83.6% service level with 64 hang-ups.



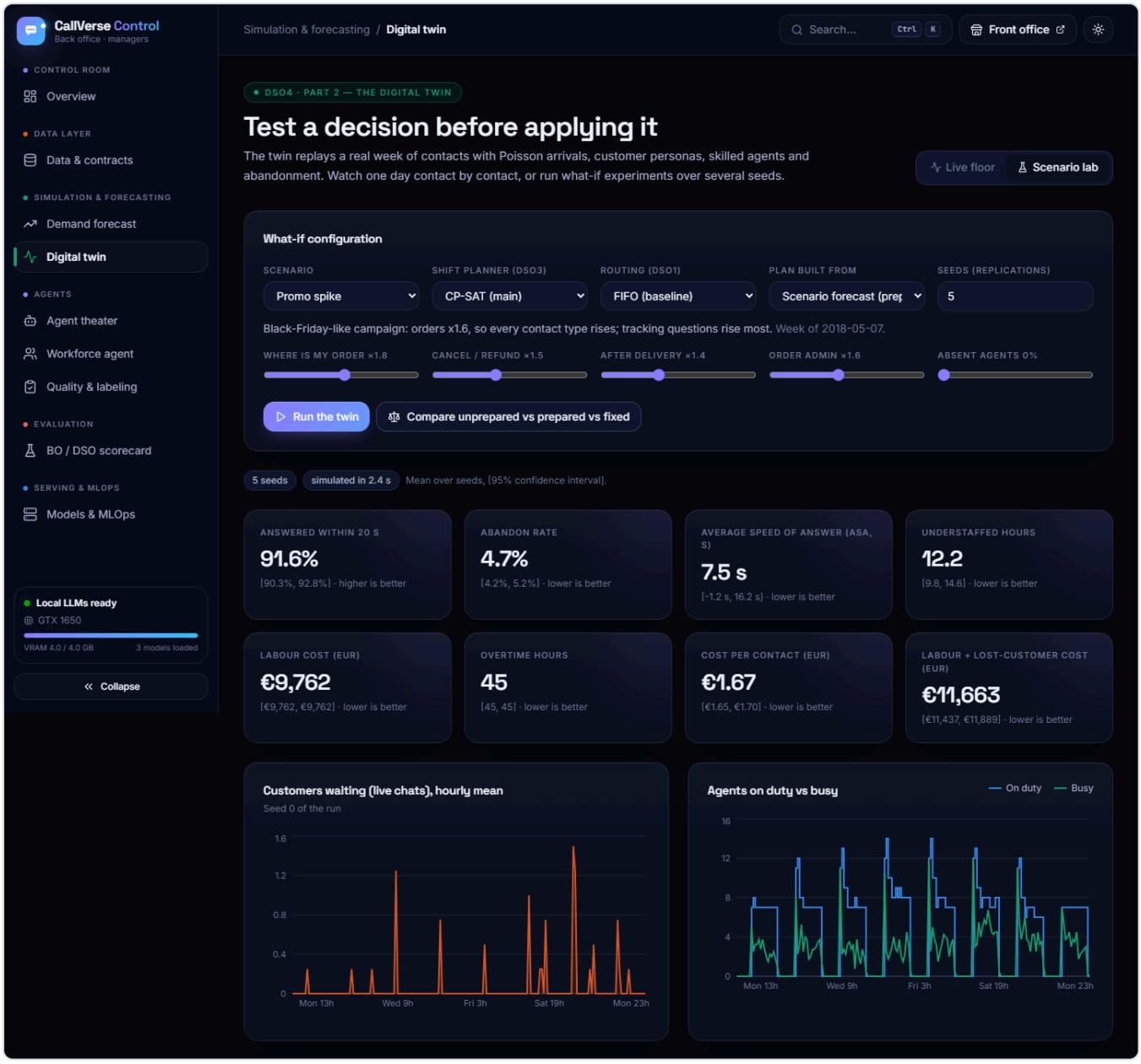
Same customers, same moment, with the plan built after testing the scenario (prepared).

What preparing changes

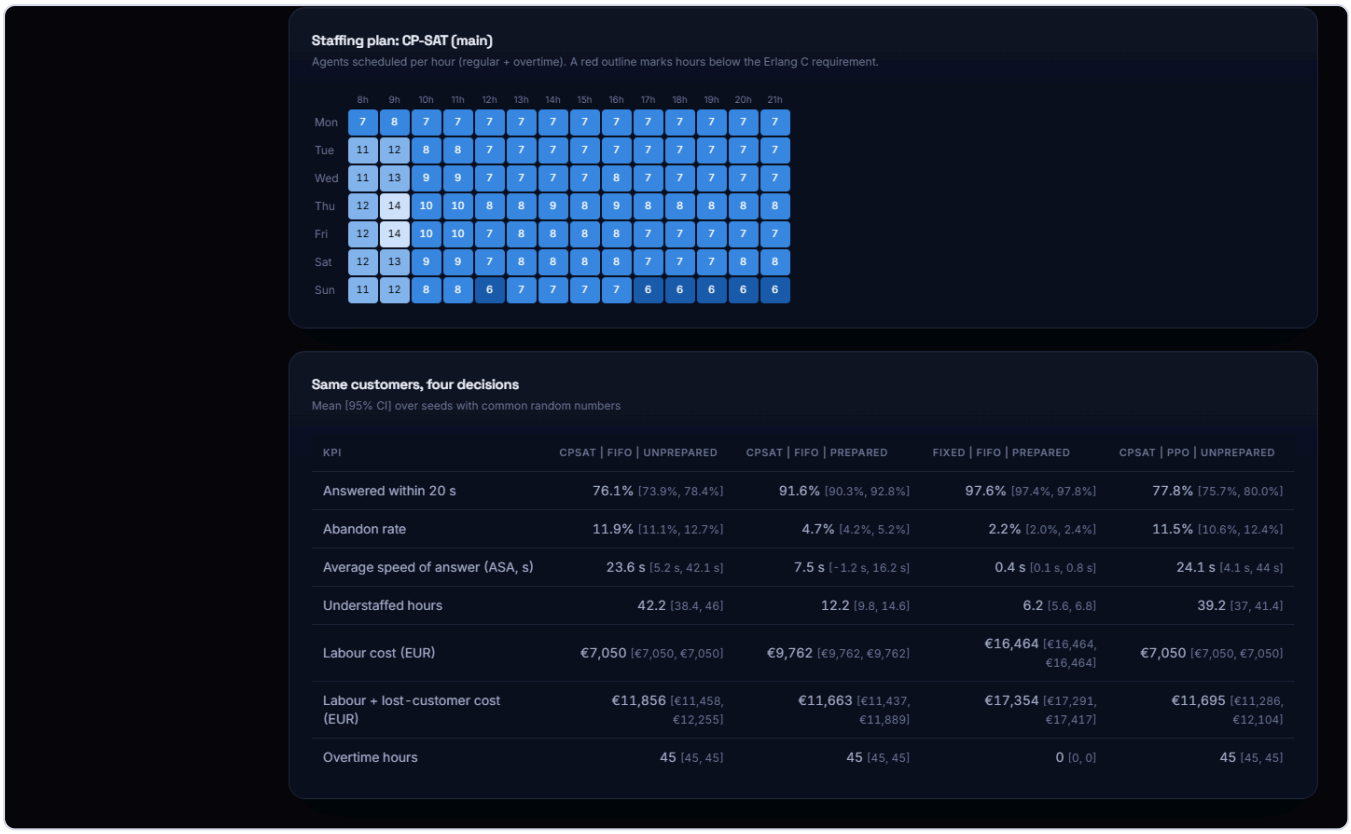
The prepared plan puts **11** agents on duty at opening instead of 8, so the overnight backlog shrinks faster (47 messages left at 08:32 instead of 78) and live customers are reached sooner. Over the whole day: **93.1%** service level and **28** hang-ups instead of 83.6% and 64. This is BO4 ("prepare for peaks by testing first") made visible; the scenario lab below measures it over several seeds with confidence intervals. It also shows why routing matters: a router that serves live customers before overnight messages (skill score, PPO) attacks the same opening problem.

4.5 Digital twin — the scenario lab

Purpose: run what-if experiments with statistics: one configuration over several seeds, then several options on the same customers.



Scenario lab: a run (promo spike, CP-SAT, FIFO, prepared, 5 seeds) and the comparison of four decisions. (part 1/2)

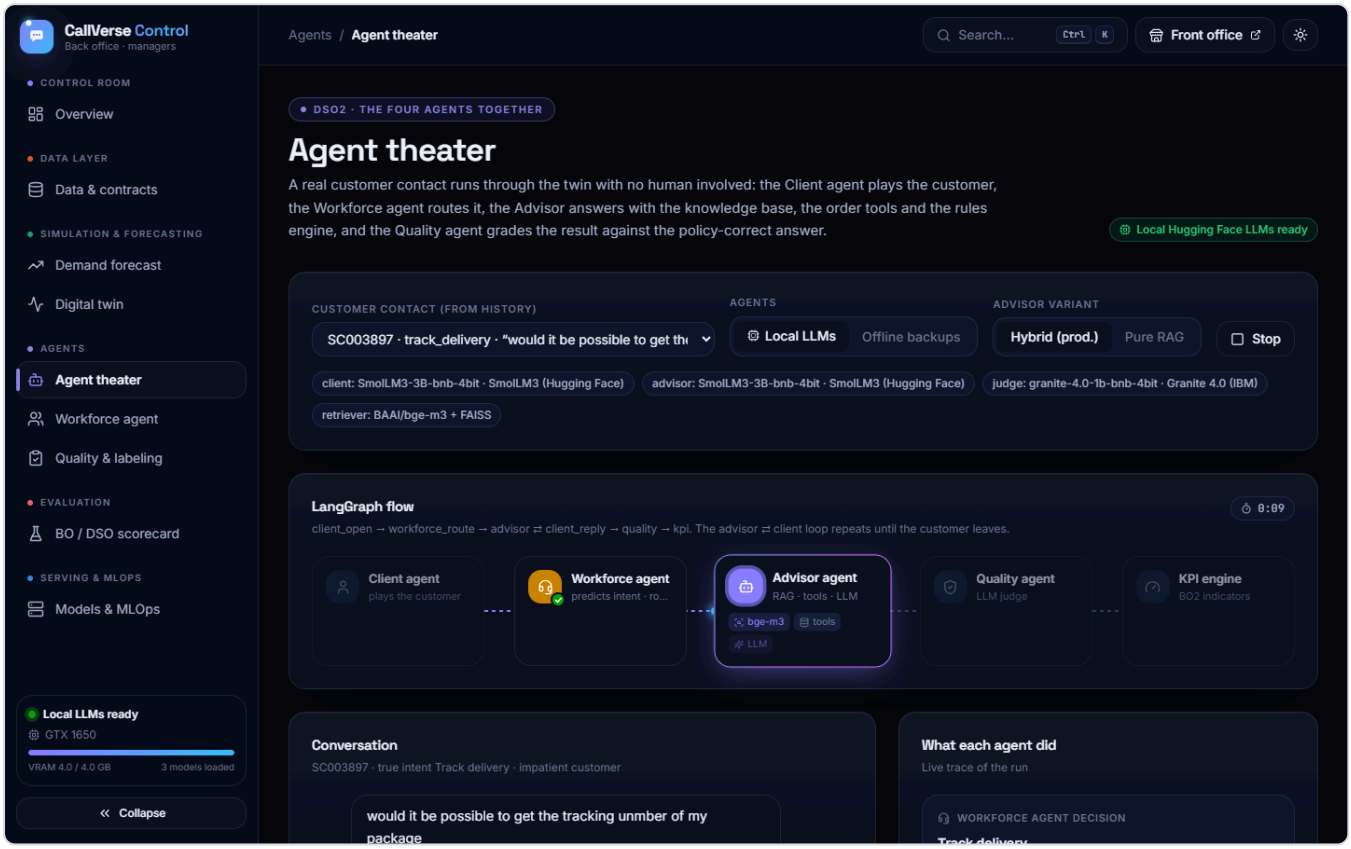


Scenario lab: a run (promo spike, CP-SAT, FIFO, prepared, 5 seeds) and the comparison of four decisions. (part 2/2)

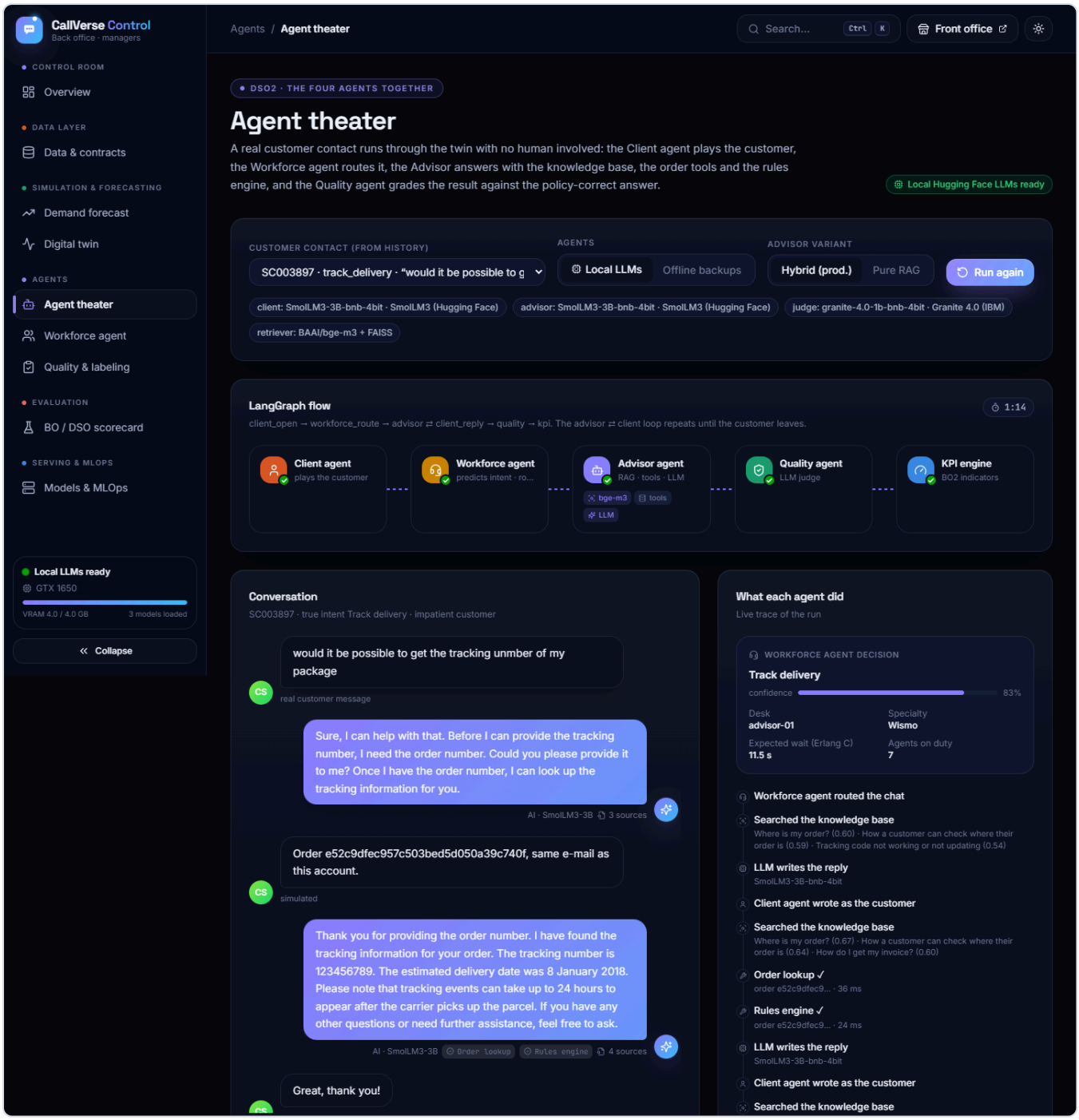
1. **What-if configuration**: scenario, planner, router, prepared/unprepared, number of seeds; sliders multiply the volume of each problem family and set the share of absent agents (custom what-ifs on top of a scenario).
2. **KPI tiles**: mean over the seeds with its **[95% confidence interval]** and whether lower or higher is better: service level, abandon rate, ASA, understaffed hours, labour cost, overtime, cost per contact, total cost.
3. **Customers waiting** (area chart): hourly mean number of live customers waiting over the simulated week (seed 0) — one hump per day.
4. **Agents on duty vs busy** (lines): the step line is the plan (agents on duty), the other line the agents actually busy; when the two touch, the centre is saturated and the queue grows.
5. **Staffing plan heatmap**: agents scheduled per day and hour (darker = more); a red outline marks hours below the Erlang C requirement.
6. **"Same customers, four decisions"**: unprepared CP-SAT, prepared CP-SAT, fixed schedule, and CP-SAT with the PPO router, compared on every KPI with common random numbers. Typical promo-spike result: preparing cuts understaffed hours by ~71% and abandonment by ~62%; the fixed schedule also reaches the target but costs much more.

4.6 Agent theater

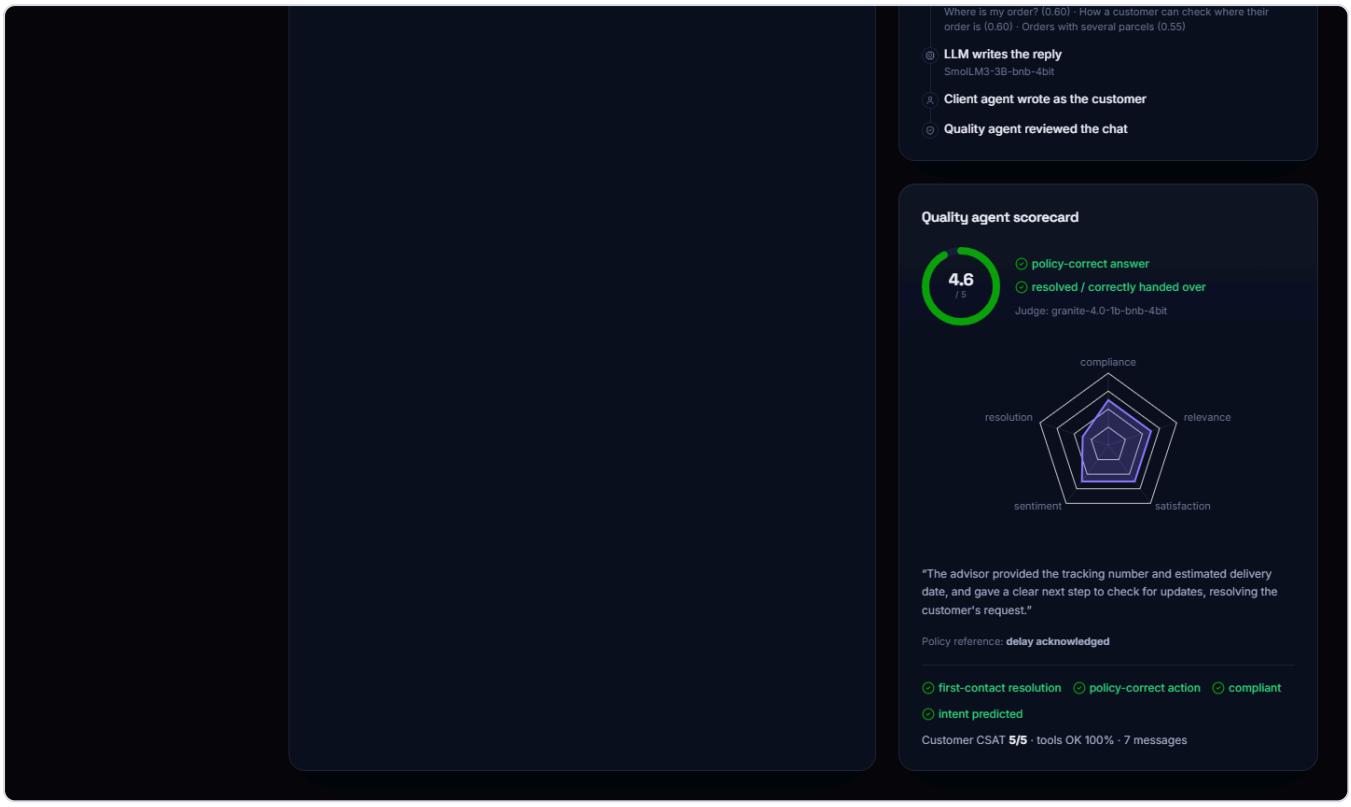
Purpose: watch the four agents handle a real contact with no human involved — this is how DSO2 batches are evaluated.



During the run: the pipeline shows the active agent and the advisor's reply streams in.



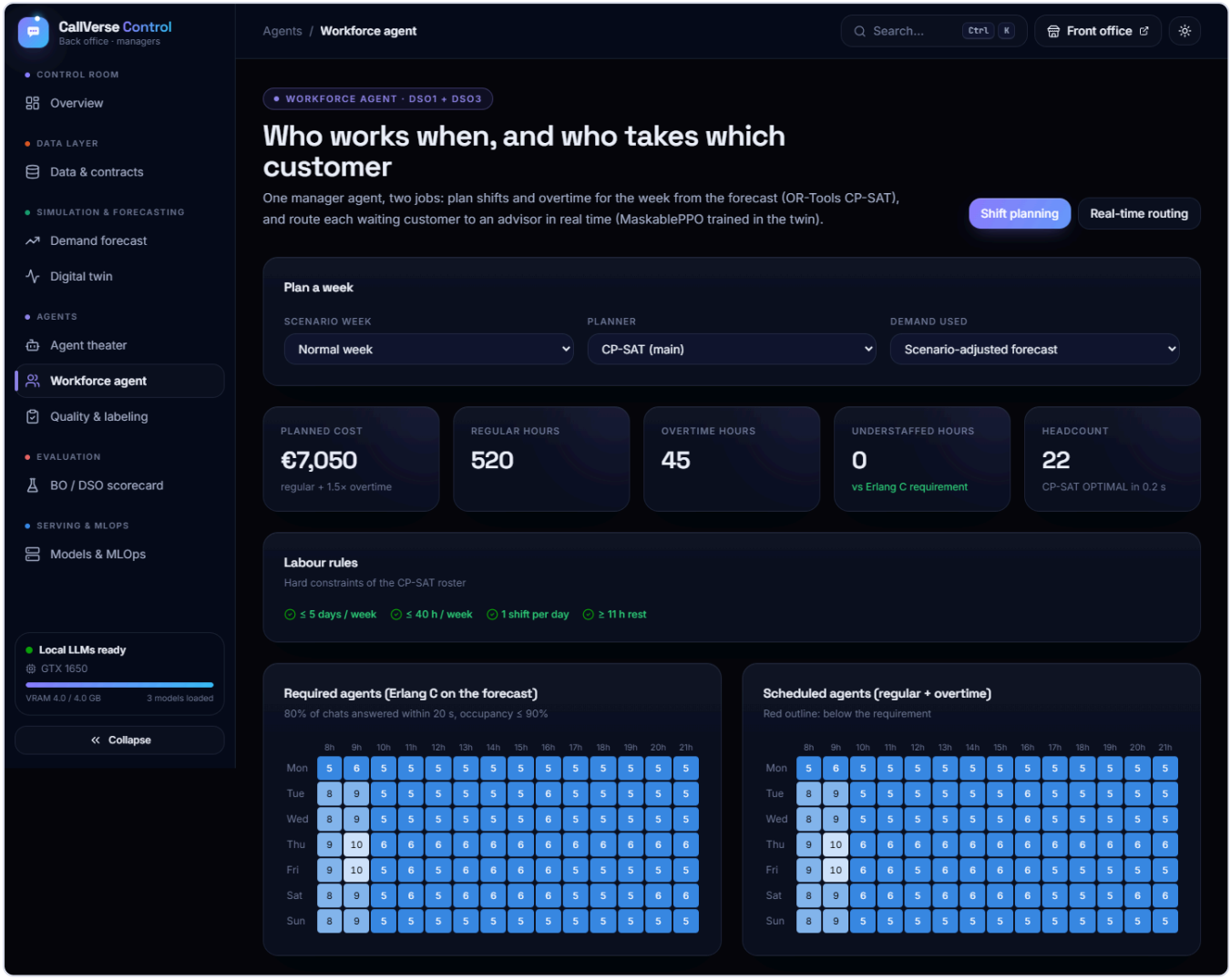
At the end: transcript, live trace and the Quality agent's scorecard. (part 1/2)



At the end: transcript, live trace and the Quality agent's scorecard. (part 2/2)

1. **Controls**: a real contact from history (or random), **Local LLMs** vs offline backups, **Hybrid** (rag_policy) vs **Pure RAG**; pills with the model of each role and the retriever.
2. **LangGraph flow**: client_open → workforce_route → advisor ⇄ client_reply → quality → kpi. The active node glows with a pulsing ring, finished nodes get a check mark, a particle travels along the edge being executed; the timer shows the elapsed time.
3. **Conversation**: the customer (Client agent) on the left, the advisor on the right, with mode, tools and sources under each message.
4. **What each agent did**: the Workforce decision and the step-by-step trace.
5. **Quality agent scorecard**: radar, verdicts and rationale; then the BO2 KPIs: first-contact resolution, policy-correct action, compliant, intent predicted, the simulated customer's CSAT, tool success rate, number of messages.

4.7 Workforce agent — shift planning



Shift planning tab: a CP-SAT week. (part 1/2)

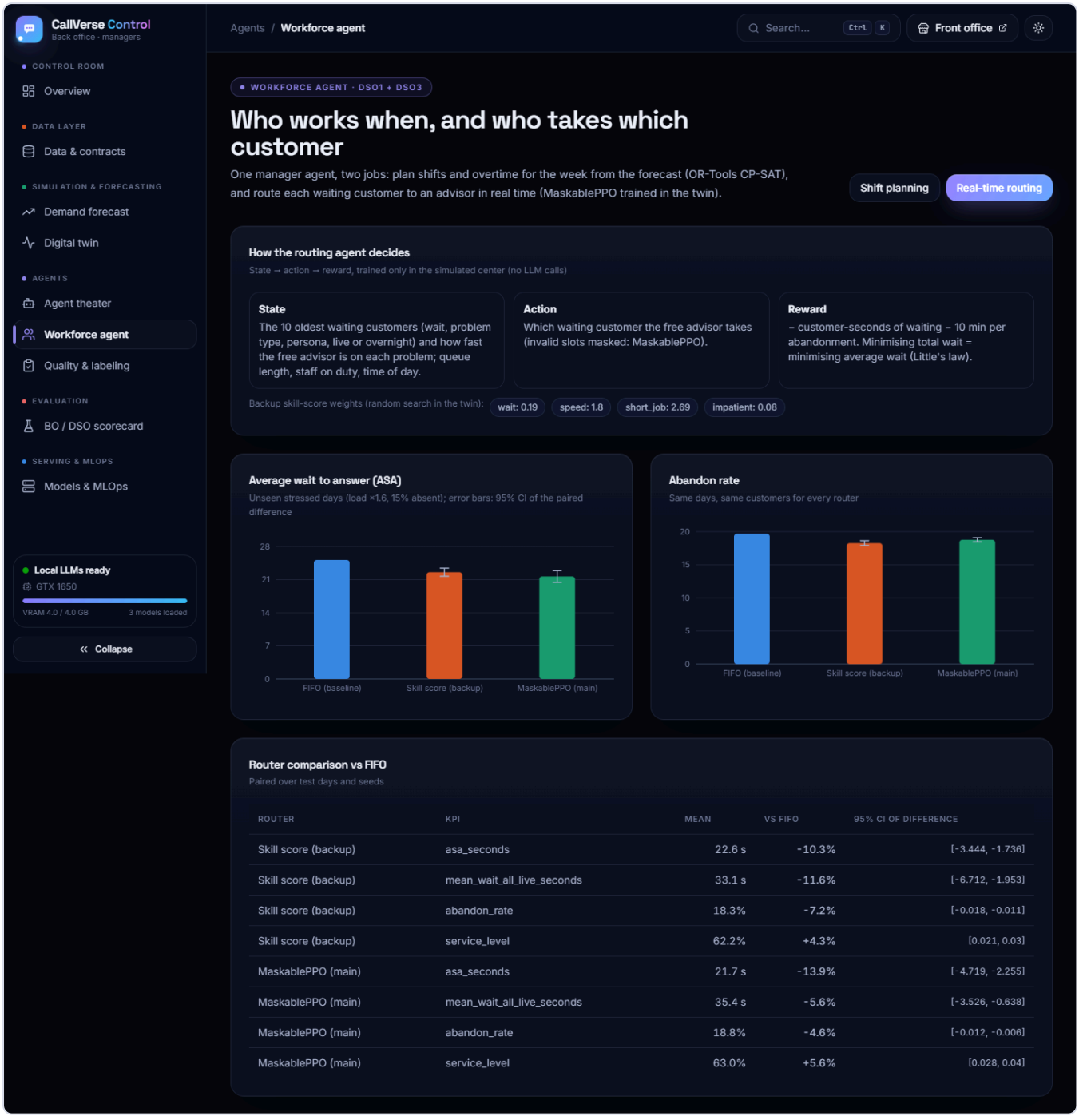


Shift planning tab: a CP-SAT week. (part 2/2)

1. **Plan a week**: scenario week, planner (CP-SAT, Erlang C + rules, fixed, fixed tuned, Erlang C hourly) and the demand used.

2. **Stat tiles**: planned cost (regular + 1.5 × overtime), regular hours, overtime hours, **understaffed hours** (hours where scheduled < required; 0 means the plan covers the requirement everywhere), headcount and solve time.
3. **Labour rules**: the four hard constraints, each checked on the roster (≤ 5 days, ≤ 40 h, one shift a day, ≥ 11 h rest).
4. **Required agents heatmap**: Erlang C requirement per day (rows) and hour (columns) for 80/20 and occupancy ≤ 90%; darker = more agents.
5. **Scheduled agents heatmap**: what the plan provides (regular + overtime); a red outline would mark an hour below the requirement. Comparing both heatmaps shows how closely CP-SAT follows the demand curve, which is where the −40% cost comes from.
6. **Roster**: each named agent's shift per day (start–end) and weekly hours.

4.8 Workforce agent — real-time routing



Real-time routing tab. (part 1/2)

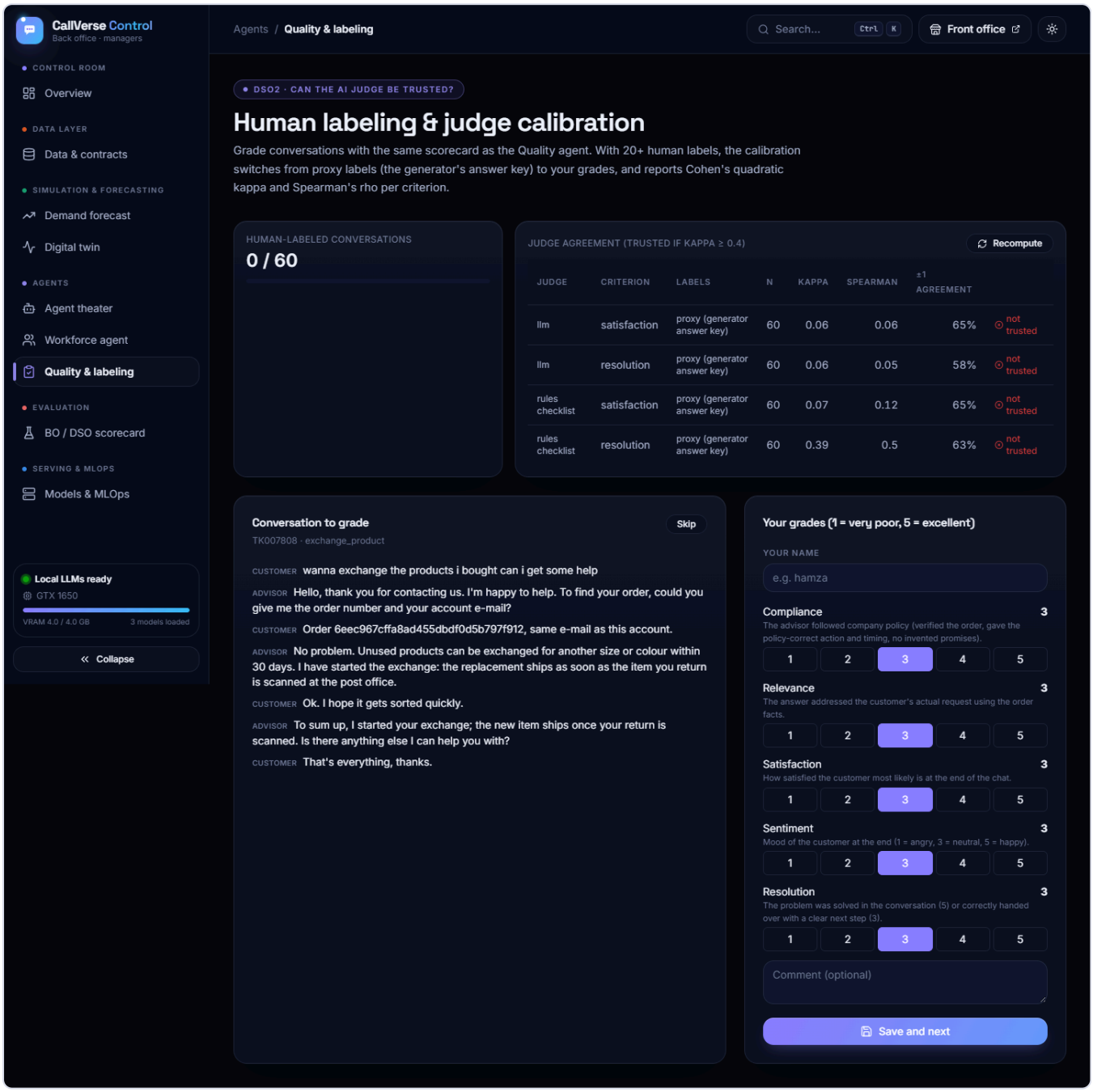


Real-time routing tab. (part 2/2)

1. **How the routing agent decides**: the state, action and reward of the MDP (section 2.6), and the tuned skill-score weights.
2. **Average wait to answer (ASA)** and **abandon rate** — bars per router on unseen stressed days (load ×1.6, 15% absent), with error bars = 95% CI of the paired difference with FIFO.
3. **Router comparison vs FIFO** — table of the relative change and its confidence interval for ASA, abandonment, service level and mean wait.
4. **PPO learning curve** — reward per simulated day (rolling mean of 50 episodes): it rises as the policy learns to reduce waiting and hang-ups, then plateaus.

4.9 Quality & labeling

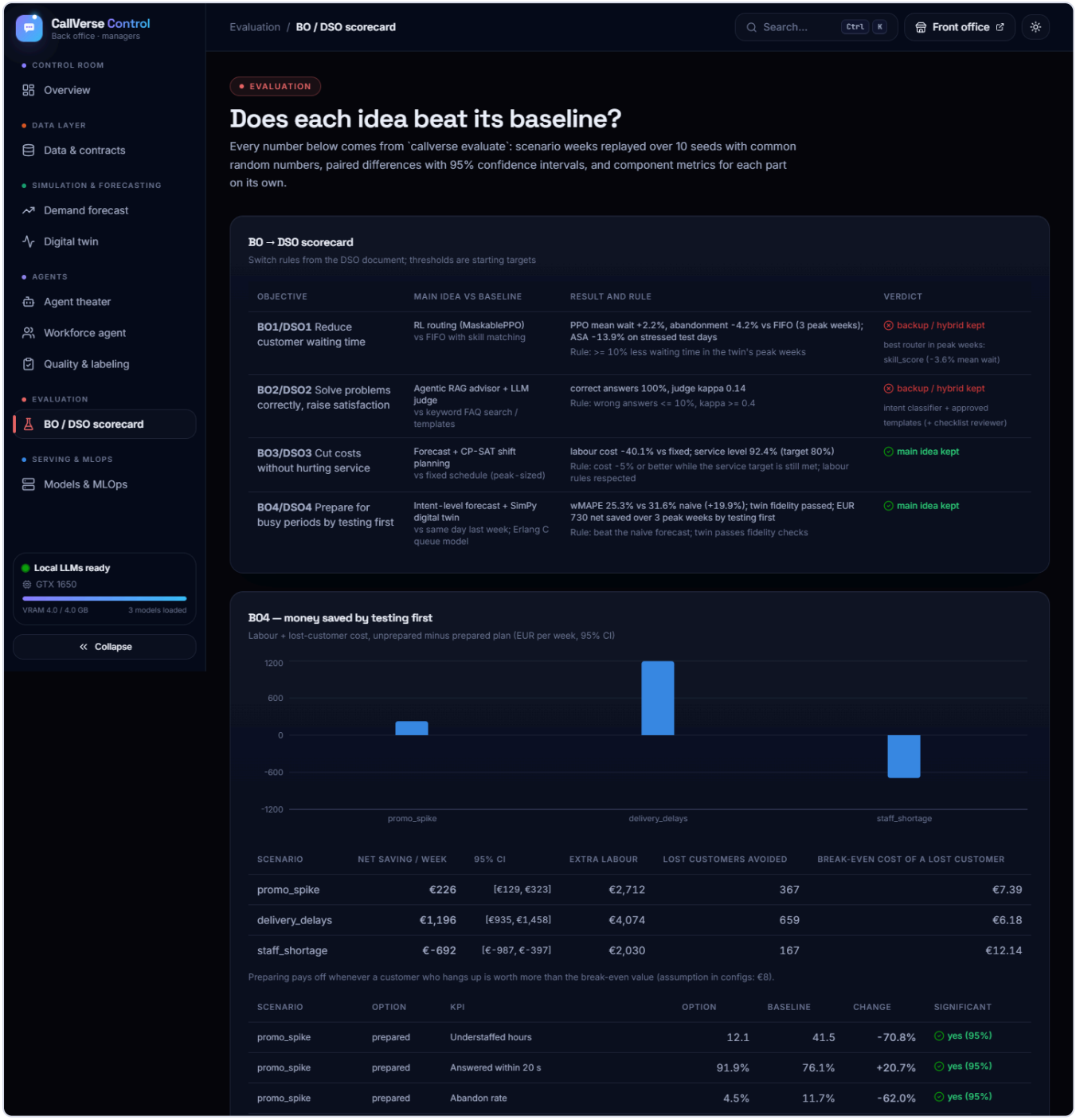
Purpose: decide whether the AI judge can be trusted, by comparing it with human grades.



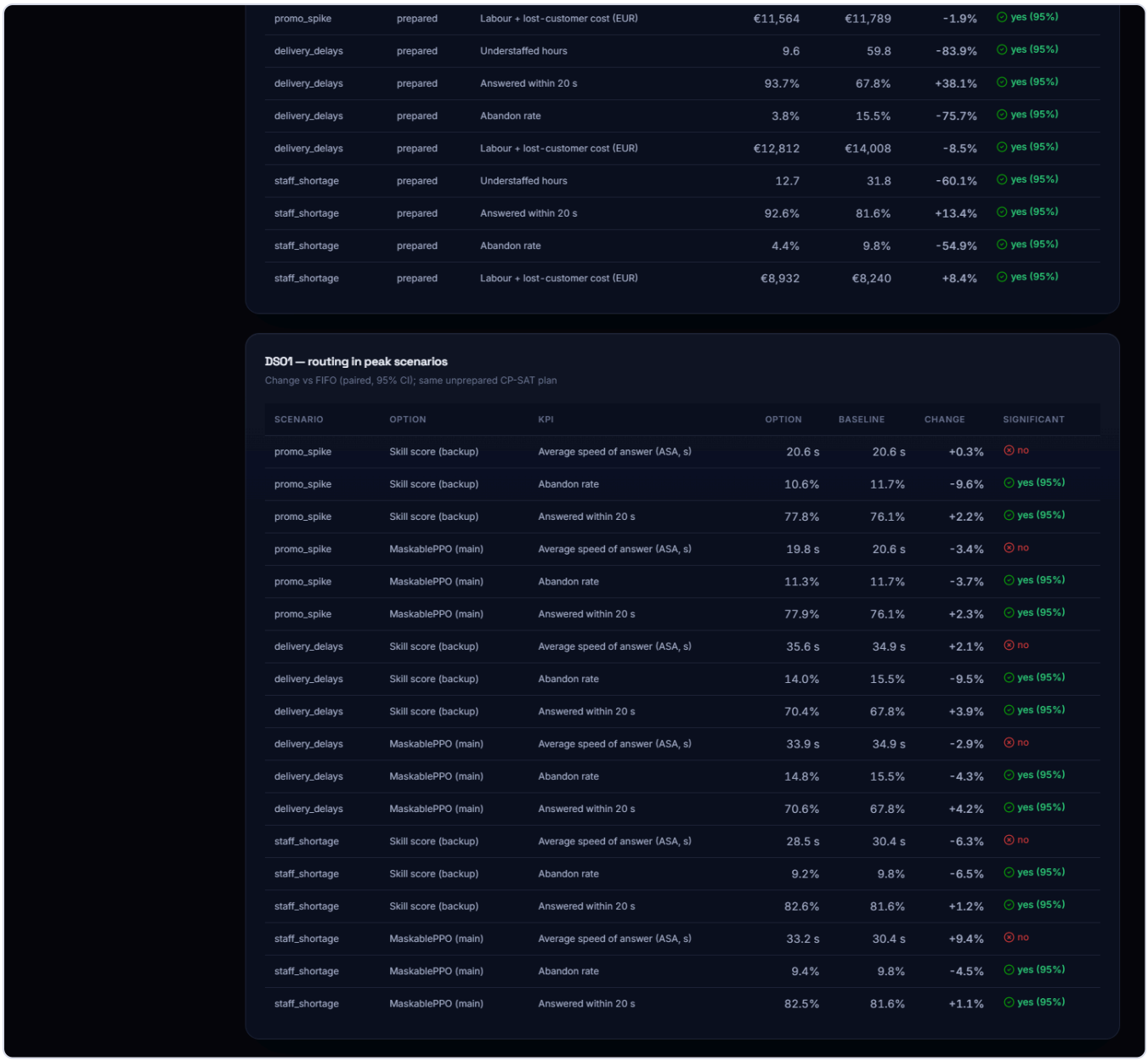
Labeling studio and judge calibration.

1. **Human-labeled conversations**: progress towards the 60 target; from 20 labels on, calibration uses human grades instead of proxy labels.
2. **Judge agreement** table: for each judge (LLM, checklist) and criterion, the label source, n, **quadratic kappa**, Spearman ρ, the share of grades within ±1 point and the verdict (trusted at kappa ≥ 0.4). "Recompute" re-runs the calibration.
3. **Conversation to grade**: a historical chat (customer and advisor turns); "Skip" draws another one.
4. **Your grades**: the same five criteria as the Quality agent, with their definitions, as 1–5 buttons; "Save and next" stores the labels in `data/labels/human_quality_labels.csv`.

4.10 BO / DSO scorecard (evaluation)



Evaluation page. (part 1/4)



promo_spike	prepared	Labour + lost-customer cost (EUR)	€11,564	€11,789	-1.9%	yes (95%)
delivery_delays	prepared	Understaffed hours	9.6	59.8	-83.9%	yes (95%)
delivery_delays	prepared	Answered within 20 s	93.7%	67.8%	+38.1%	yes (95%)
delivery_delays	prepared	Abandon rate	3.8%	15.5%	-75.7%	yes (95%)
delivery_delays	prepared	Labour + lost-customer cost (EUR)	€12,812	€14,008	-8.5%	yes (95%)
staff_shortage	prepared	Understaffed hours	12.7	31.8	-60.1%	yes (95%)
staff_shortage	prepared	Answered within 20 s	92.6%	81.6%	+13.4%	yes (95%)
staff_shortage	prepared	Abandon rate	4.4%	9.8%	-54.9%	yes (95%)
staff_shortage	prepared	Labour + lost-customer cost (EUR)	€8,932	€8,240	+8.4%	yes (95%)

DS01 — routing in peak scenarios

Change vs FIFO (paired, 95% CI); same unprepared CP-SAT plan

SCENARIO	OPTION	KPI	OPTION	BASELINE	CHANGE	SIGNIFICANT
promo_spike	Skill score (backup)	Average speed of answer (ASA, s)	20.6 s	20.6 s	+0.3%	no
promo_spike	Skill score (backup)	Abandon rate	10.6%	11.7%	-9.6%	yes (95%)
promo_spike	Skill score (backup)	Answered within 20 s	77.8%	76.1%	+2.2%	yes (95%)
promo_spike	MaskablePPO (main)	Average speed of answer (ASA, s)	19.8 s	20.6 s	-3.4%	no
promo_spike	MaskablePPO (main)	Abandon rate	11.3%	11.7%	-3.7%	yes (95%)
promo_spike	MaskablePPO (main)	Answered within 20 s	77.9%	76.1%	+2.3%	yes (95%)
delivery_delays	Skill score (backup)	Average speed of answer (ASA, s)	35.6 s	34.9 s	+2.1%	no
delivery_delays	Skill score (backup)	Abandon rate	14.0%	15.5%	-9.5%	yes (95%)
delivery_delays	Skill score (backup)	Answered within 20 s	70.4%	67.8%	+3.9%	yes (95%)
delivery_delays	MaskablePPO (main)	Average speed of answer (ASA, s)	33.9 s	34.9 s	-2.9%	no
delivery_delays	MaskablePPO (main)	Abandon rate	14.8%	15.5%	-4.3%	yes (95%)
delivery_delays	MaskablePPO (main)	Answered within 20 s	70.6%	67.8%	+4.2%	yes (95%)
staff_shortage	Skill score (backup)	Average speed of answer (ASA, s)	28.5 s	30.4 s	-6.3%	no
staff_shortage	Skill score (backup)	Abandon rate	9.2%	9.8%	-6.5%	yes (95%)
staff_shortage	Skill score (backup)	Answered within 20 s	82.6%	81.6%	+1.2%	yes (95%)
staff_shortage	MaskablePPO (main)	Average speed of answer (ASA, s)	33.2 s	30.4 s	+9.4%	no
staff_shortage	MaskablePPO (main)	Abandon rate	9.4%	9.8%	-4.5%	yes (95%)
staff_shortage	MaskablePPO (main)	Answered within 20 s	82.5%	81.6%	+1.1%	yes (95%)

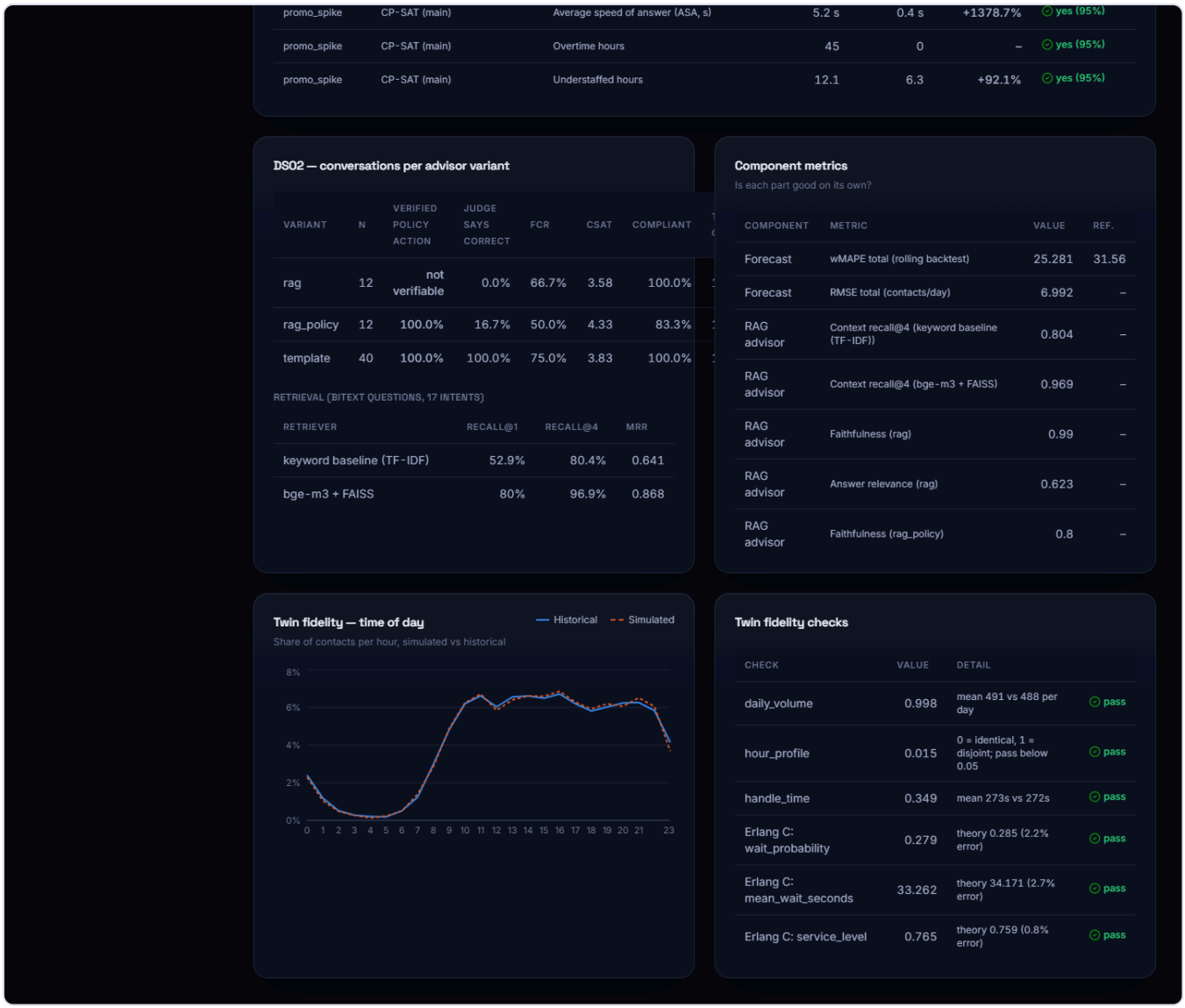
Evaluation page. (part 2/4)

DS03 — shift planners vs the fixed schedule

Same customers; prepared plans; FIFO routing

SCENARIO	OPTION	KPI	OPTION	BASELINE	CHANGE	SIGNIFICANT
baseline	Fixed, tuned (sensitivity)	Labour cost (EUR)	€7,056	€11,760	-40.0%	yes (95%)
baseline	Fixed, tuned (sensitivity)	Cost per contact (EUR)	€1.96	€3.20	-38.5%	yes (95%)
baseline	Fixed, tuned (sensitivity)	Answered within 20 s	93.1%	97.7%	-4.8%	yes (95%)
baseline	Fixed, tuned (sensitivity)	Average speed of answer (ASA, s)	11.6 s	0.4 s	+2488.0%	no
baseline	Fixed, tuned (sensitivity)	Overtime hours	0	0	–	no
baseline	Fixed, tuned (sensitivity)	Understaffed hours	9.8	6.2	+58.1%	yes (95%)
baseline	Erlang C + rules (backup)	Labour cost (EUR)	€9,792	€11,760	-16.7%	yes (95%)
baseline	Erlang C + rules (backup)	Cost per contact (EUR)	€2.68	€3.20	-16.1%	yes (95%)
baseline	Erlang C + rules (backup)	Answered within 20 s	95.8%	97.7%	-2.0%	yes (95%)
baseline	Erlang C + rules (backup)	Average speed of answer (ASA, s)	12 s	0.4 s	+2579.8%	no
baseline	Erlang C + rules (backup)	Overtime hours	0	0	–	no
baseline	Erlang C + rules (backup)	Understaffed hours	8.1	6.2	+30.6%	yes (95%)
baseline	CP-SAT (main)	Labour cost (EUR)	€7,050	€11,760	-40.1%	yes (95%)
baseline	CP-SAT (main)	Cost per contact (EUR)	€1.95	€3.20	-38.9%	yes (95%)
baseline	CP-SAT (main)	Answered within 20 s	92.4%	97.7%	-5.4%	yes (95%)
baseline	CP-SAT (main)	Average speed of answer (ASA, s)	13.6 s	0.4 s	+2937.3%	no
baseline	CP-SAT (main)	Overtime hours	45	0	–	yes (95%)
baseline	CP-SAT (main)	Understaffed hours	10.9	6.2	+75.8%	yes (95%)
promo_spike	Fixed, tuned (sensitivity)	Labour cost (EUR)	€9,408	€16,464	-42.9%	yes (95%)
promo_spike	Fixed, tuned (sensitivity)	Cost per contact (EUR)	€1.63	€2.77	-40.9%	yes (95%)
promo_spike	Fixed, tuned (sensitivity)	Answered within 20 s	91.3%	97.7%	-6.5%	yes (95%)
promo_spike	Fixed, tuned (sensitivity)	Average speed of answer (ASA, s)	4.5 s	0.4 s	+1170.4%	yes (95%)
promo_spike	Fixed, tuned (sensitivity)	Overtime hours	0	0	–	no
promo_spike	Fixed, tuned (sensitivity)	Understaffed hours	14	6.3	+122.2%	yes (95%)
promo_spike	Erlang C + rules (backup)	Labour cost (EUR)	€13,440	€16,464	-18.4%	yes (95%)
promo_spike	Erlang C + rules (backup)	Cost per contact (EUR)	€2.28	€2.77	-17.7%	yes (95%)
promo_spike	Erlang C + rules (backup)	Answered within 20 s	95.4%	97.7%	-2.3%	yes (95%)
promo_spike	Erlang C + rules (backup)	Average speed of answer (ASA, s)	1.3 s	0.4 s	+263.0%	yes (95%)
promo_spike	Erlang C + rules (backup)	Overtime hours	0	0	–	no
promo_spike	Erlang C + rules (backup)	Understaffed hours	8.7	6.3	+38.1%	yes (95%)
promo_spike	CP-SAT (main)	Labour cost (EUR)	€9,762	€16,464	-40.7%	yes (95%)
promo_spike	CP-SAT (main)	Cost per contact (EUR)	€1.67	€2.77	-39.5%	yes (95%)
promo_spike	CP-SAT (main)	Answered within 20 s	91.9%	97.7%	-5.9%	yes (95%)

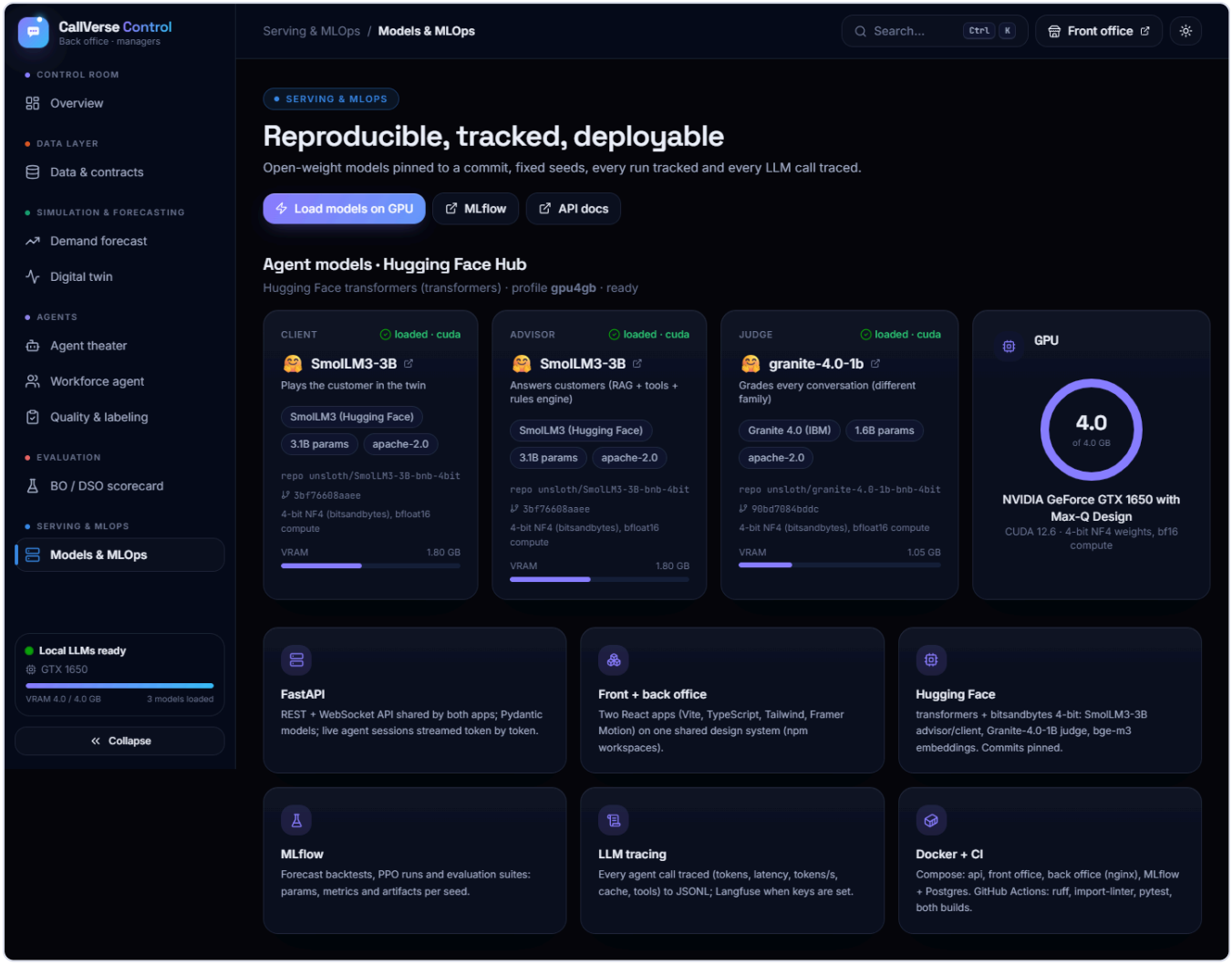
Evaluation page. (part 3/4)



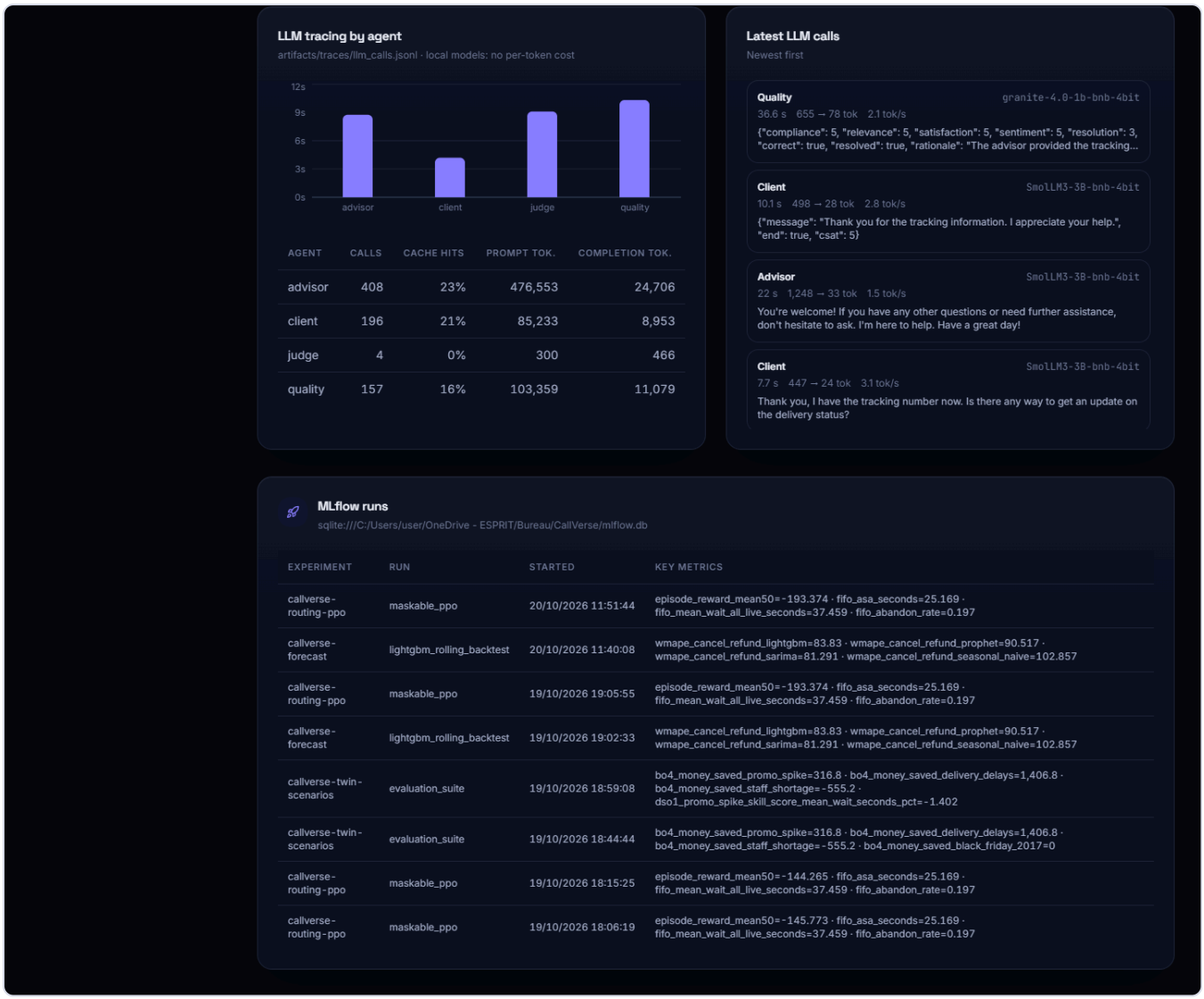
Evaluation page. (part 4/4)

1. **Scorecard table**: each objective, main idea vs baseline, the measured result, the switch rule and the verdict (main idea kept / backup or hybrid kept, with the fallback used).
2. **BO4 — money saved by testing first**: bars of the net weekly saving per scenario (labour + lost customers, unprepared minus prepared, 95% CI); the table adds the extra labour, the lost customers avoided and the **break-even** value of a lost customer. A negative saving (staff shortage) means preparing cost more labour than the €8 assumption recovers.
3. **Paired difference tables** (DSO1, DSO3, BO4): option mean, baseline mean, relative change and "significant (95%)" when the CI excludes 0.
4. **DSO2**: conversations per advisor variant (verified policy action, judge verdict, FCR, CSAT, compliance, tools OK) and retrieval quality (recall@1, recall@4, MRR) of keyword search vs bge-m3.
5. **Component metrics**: each part on its own with its reference value.
6. **Twin fidelity**: hourly share of contacts, simulated (dashed) vs historical (solid) — the two curves overlap — and the checks table (KS p-values, distances, Erlang C errors) with pass/fail.

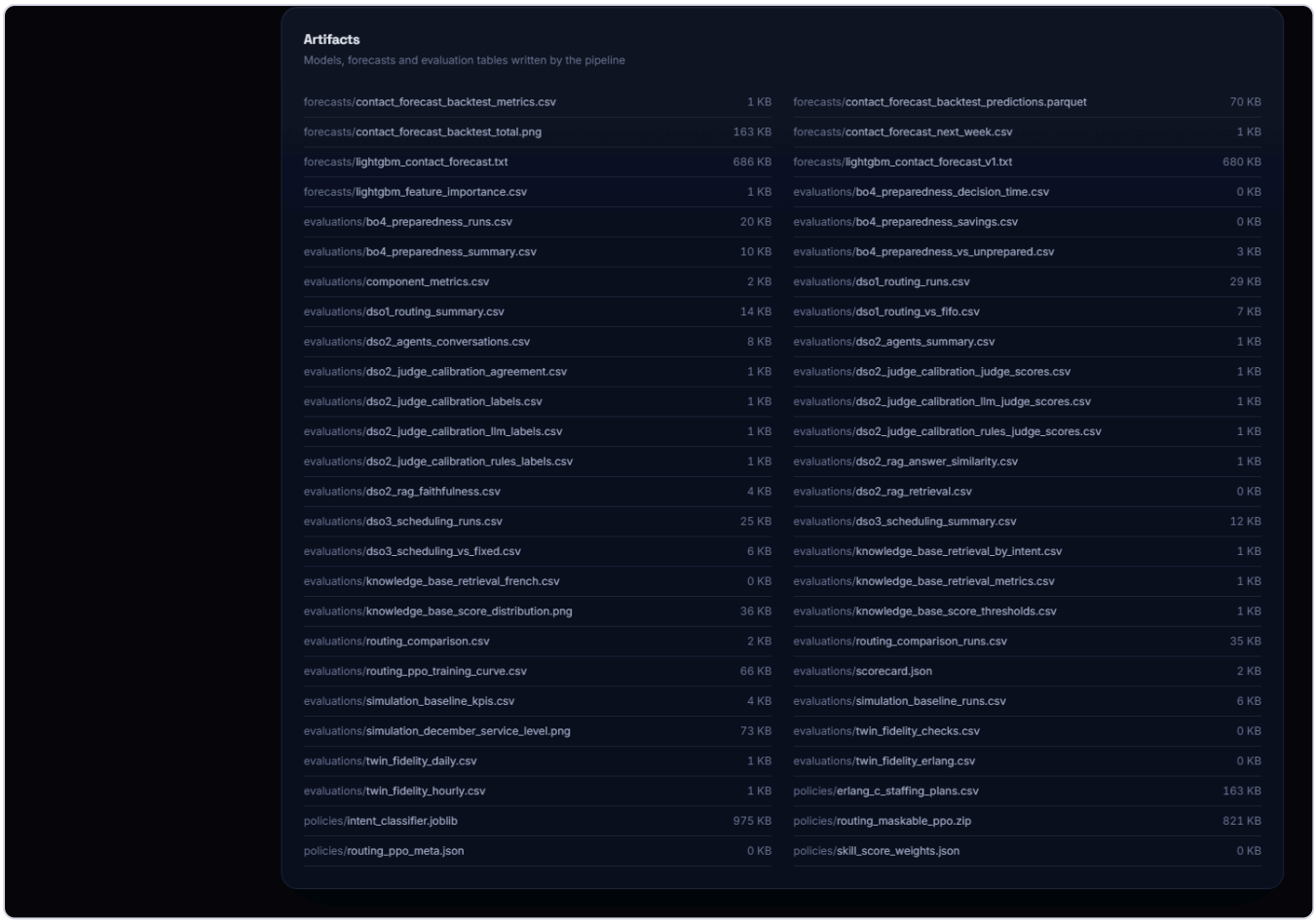
4.11 Models & MLOps



Models and MLOps page. (part 1/3)



Models and MLOps page. (part 2/3)



Models and MLOps page. (part 3/3)

1. **Load models on GPU**: loads every role's model in advance so the first chat is fast; links to MLflow and the API documentation.
2. **Model cards** (one per role): the Hugging Face base model (link to the Hub), family, parameters, licence, the exact repository and pinned commit, the quantization, the status (downloaded / loaded on cuda) and the VRAM footprint bar.
3. **GPU gauge**: memory used of the 4 GB, GPU name, CUDA version.
4. **Stack cards**: FastAPI, the two apps, Hugging Face, MLflow, LLM tracing, Docker + CI.
5. **LLM tracing by agent**: mean latency per agent (bars) and a table of calls, cache-hit share and tokens.
6. **Latest LLM calls**: newest first, with latency, prompt → completion tokens, tokens per second and a preview of the output.
7. **MLflow runs** and **artifacts** written by the pipeline (forecasts, policies, evaluation tables).

PART 5

End-to-end walkthroughs

Two journeys through the whole system, step by step.

5.1 One customer message, from keyboard to quality review

#	Where	What happens
1	Front office (React)	The chat page opens a WebSocket to `/ws/session` and sends `start` with the demo customer's contact id, customer = human, advisor = agent.
2	API → `LiveSession`	Builds the session: customer profile from the contact (order, timestamp, true intent), the agents, the tools "as of" the contact time; replies `ready`.
3	Browser	The customer sends a message → `customer_message`.
4	Workforce agent	First message only: intent classifier → problem family → best-skilled desk; Erlang C expected wait for that hour → `routing` event.
5	Advisor — retrieval	bge-m3 embeds the customer's words, FAISS returns the top-3 policy sections → `stage: retrieval`.
6	Advisor — verification	No order number yet → only the knowledge-base tool is offered; the LLM is told to ask for the order number.
7	LLM (GPU)	SmolLM3 renders its chat template, generates; tokens stream through the stream gate → `token` events → the bubble types itself.
8	Browser	The customer gives the order number.
9	Advisor — rules engine	`lookup_order` reads SQLite as of the contact time; `recommend_resolution` applies the policy (e.g. tracking shared; carrier investigation if late with no movement) → `stage: tool` events.
10	LLM	Receives the decision and the approved answer; writes the reply in natural language (it cannot change the decision).
11	Browser	"End chat" → stars → `end` with the CSAT.
12	Quality agent	Granite judges the transcript against the policy reference → scorecard; the KPI row is computed; the transcript is saved → `review`, `done`.

5.2 One what-if, from click to confidence interval

1. The manager picks *promo spike*, CP-SAT, FIFO, *unprepared* in the scenario lab and clicks Run.
2. The API builds the week's demand: the forecast for that week (normal forecast for an unprepared plan, scenario-adjusted for a prepared one).
3. Erlang C converts the hourly demand into the hourly requirement (80/20, occupancy ≤ 90%).
4. CP-SAT chooses the shifts and builds the roster under the labour rules; the plan becomes a duty matrix.
5. For each seed, the twin draws the *real* scenario customers (with the scenario multipliers and absences), then SimPy replays the week with the router, the overtime rule and the patience of every customer.
6. KPIs per seed → means and 95% confidence intervals; for "compare", the same seeds are reused for every option (common random numbers) and the paired differences decide significance.
7. The page shows the tiles, the waiting and staffing charts and the plan heatmap; the live floor can replay any single day of it.

Appendix

A. Glossary

Term	Meaning
ASA	Average speed of answer: mean wait of the customers who were answered.
Abandonment	A customer leaves the queue before being answered.
Service level 80/20	Share of contacts answered within 20 seconds; the target is 80%.
Occupancy	Share of on-duty time advisors spend handling contacts.
FCR	First-contact resolution: solved in the first conversation, with no hand-over.
CSAT	Customer satisfaction score, 1–5, from the post-chat survey.
Erlang C	Queueing formula giving the probability of waiting with c advisors and load A.
Discrete-event simulation	A simulation that jumps from one event to the next (SimPy).
Common random numbers	Every option is simulated on the same random customers, so differences come from the decision only.
Paired 95% CI	Confidence interval of the per-seed difference between two options.
wMAPE	Σ|error| / Σ actual: forecast error weighted by volume.
Rolling-origin backtest	Repeated forecasts where each model only sees data before the forecast date.
CP-SAT	Google OR-Tools constraint-programming / SAT solver for integer optimisation.
MaskablePPO	Proximal Policy Optimization with action masking (invalid actions can never be chosen).
RAG	Retrieval-augmented generation: the LLM answers from retrieved documents.
Embedding / cosine similarity	A text turned into a vector; two texts with similar meaning have vectors with a high cosine.
FAISS	Library for fast vector search (exact inner-product index here).
Quantization (NF4)	Storing model weights in 4 bits with levels matched to a normal distribution.
bfloat16	16-bit floating format with the exponent range of float32 (no overflow where float16 overflows).
LLM-as-judge	An LLM grading another model's output against a rubric.
Cohen's quadratic kappa	Chance-corrected agreement on an ordinal scale; ≥ 0.4 = moderate agreement.
Kolmogorov–Smirnov test	Tests whether two samples come from the same distribution.
LangGraph	Library to run agents as an explicit graph / state machine.
WebSocket	A persistent two-way connection: the server pushes events (tokens, steps) to the browser.

B. Commands

Command	Purpose
`callverse all [--quick] [--skip-data]`	Whole pipeline: data → forecast → RL → evaluation.
`callverse llm-pull` / `llm-pin` / `status`	Download and pin the Hugging Face models; check GPU, data and models.
`callverse evaluate`	Every experiment, component metrics and the scorecard.
`callverse agents --n 12 --variants rag_policy,rag`	DSO2 conversation batches with the LLMs.
`callverse judge-calibrate --judge llm`	Agreement of the LLM judge with the labels.
`callverse serve`	API for both apps (Swagger at /docs).
`npm run dev:front` / `npm run dev:back` (in `frontend/`)	Front office :5173 / back office :5174.
`docker compose up --build`	Everything in containers (add `-f docker-compose.gpu.yml` for the GPU).

C. Folder map

Folder	Content
`configs/`	All assumptions: data, simulation, agents (LLM profiles), scenarios, app.
`src/callverse/data/`	Notebook runner, Pandera contracts, loaders, SQLite store, ticket and chat generators.
`src/callverse/forecasting/`	Features, baselines, LightGBM, rolling backtest, next-week forecast.
`src/callverse/simulation/`	SimPy engine, routers, Erlang C, customers, twin setup.
`src/callverse/agents/`	`client/`, `advisor/` (retrieval, tools, classifier), `quality/`, `workforce/` (CP-SAT, PPO).
`src/callverse/llm/`	Hugging Face client: 4-bit loading, chat templates, tool parsing, streaming, cache, tracing.
`src/callverse/orchestration/`	LangGraph graph, live sessions, shared runtime.
`src/callverse/twin/`	Scenarios, scenario runner, live-floor replay, fidelity checks.
`src/callverse/evaluation/`	KPI engine, experiments, component metrics, scorecard.
`src/callverse/api/`	FastAPI routes and WebSockets.
`frontend/`	`shared/` design system, `front-office/`, `back-office/`, Dockerfile + nginx.
`tests/`, `docs/`	Unit, smoke and integration tests; report, run guide, demo script, recruiter kit, this guide.

D. Honest limits

- Contacts, tickets and chats are synthetic (generated from real orders and real wording with explicit assumptions); real logs would replace them without changing the pipeline.
- Small 3B/1.6B models on a laptop: the hybrid design is mandatory, and the LLM judge is not yet trusted (kappa < 0.4); 60 human labels are the next step.
- PPO is trained on single days; multi-day episodes are needed to beat FIFO on mean wait in full peak weeks.
- The DSO2 conversation numbers in the report predate the switch to Hugging Face models; the re-run is planned.